Team ID: 26FSPDSCPMID4098

Case-Study Report

On "Digital Thread Analytics: Programming Frameworks for End-to-End Data Science Lifecycle Management"

submitted as part of

Programming for Data Science (MACSE502)

by



REG.NO:26MML0045
NAME:VEDANT NIMKAR

DM

REG.NO:26MML0055
NAME:DURVESH MAHAJAN



REG.NO:26MML0043
NAME:AYUSH SHARMA

SP

REG.NO:26MML0022
NAME:SHREYAN PANDA

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

Index

Sl. No.	Section Topic	Page No.
	Executive Summary and Abstract	6
1.	I. Introduction	6
2.	II. Problem Statement	8
3.	III. Background and Literature Review	9
4.	IV. Requirements Analysis for Digital Thread Analytics	11
5.	V. Digital Thread Analytics Architecture	12
6.	VI. Data Acquisition and Integration Layer	13
7.	VII. Digital Thread Metadata and Knowledge Layer	14
8.	VIII. Data Engineering and Transformation Layer	16
9.	IX. Analytics and Machine Learning Lifecycle Layer	17
10.	X. Digital Thread Traceability and Provenance Layer	19
11.	XI. Workflow Orchestration and Automation Layer	19
12.	XII. Explainability, Governance and Compliance Layer	20
13.	XIII. Intelligent Decision Support Layer	21
14.	XIV. Experimental Setup	24
15.	XV. Evaluation Metrics	25
16.	XVI. Results and Findings	26
17.	XVII. Comparative Analysis	29
18.	XVIII. Reliability, Reproducibility and Auditability Assessment	30
19.	XIX. Industry Use Cases and Case Study Evaluation	31
20.	XX. Business Impact Assessment	32
21.	XXI. Challenges and Limitations	33
22.	XXII. Future Research Directions	34
23.	XXIII. Conclusion	35
	XXIV. Data Availability Statement	36
	References	36

List of Figures

Fig. No.	Figure Title	Page No.
1.	Evolution of Data Science Lifecycle Management	7
2.	Digital Thread Ecosystem Overview	7
3.	Data Science Lifecycle Challenges Framework	8
4.	Literature Taxonomy Framework	10
5.	Requirements Engineering Framework	11
6.	Digital Thread Analytics Architecture	12
7.	Layered System Architecture	13
8.	Data Acquisition Framework	14
9.	Metadata and Lineage Architecture	15
10.	Knowledge Graph for Lifecycle Traceability	16
11.	Data Engineering and Transformation Workflow	17
12.	Machine Learning Lifecycle Architecture	18
13.	Analytics Development Pipeline	18
14.	End-to-End Digital Thread Traceability Model	19
15.	Workflow Orchestration Framework	20
16.	DataOps-MLOps Integration Model	20
17.	Governance and Explainability Framework	21
18.	Decision Intelligence Framework	22
19.	Analytics-to-Decision Pipeline	22
20.	Experimental Testbed Architecture	24
21.	Evaluation Framework	25
22.	Traceability Performance Results	26
23.	Lifecycle Efficiency Analysis	27
24.	Governance Effectiveness Analysis	27
25.	Comparative Benchmark Analysis	29
26.	Reproducibility Assessment Model	30
27.	Smart Manufacturing Digital Thread Ecosystem	31
28.	Aerospace Lifecycle Analytics Architecture	32
29.	Business Impact Assessment Framework	33
30.	Challenges and Limitations Roadmap	33
31.	Future Research Roadmap	34
32.	Next-Generation Digital Thread Analytics Vision	35
33.	Research Contribution Framework	35
34.	Case Study Model Performance	28
35.	Prediction Service and Lineage Explorer	23

List of Tables

Table. No.	Table Title	Page No.
1.	Executive summary of contributions and outcomes	7
2.	Research challenges and the corresponding gaps in current practice	9
3.	Comparative analysis of existing lifecycle management approaches	10
4.	Functional and non-functional requirements matrix	11
5.	Architectural layers and their contribution to the thread	13
6.	Data source classification and identity strategy	14
7.	Provenance and traceability attributes of the ledger vocabulary	16
8.	Data transformation components	17
9.	Governance controls and their implementing mechanism	21
10.	Experimental configuration	24
11.	Metrics definition matrix	26
12.	Experimental results summary	28
13.	Comparative benchmark analysis	29
14.	Lifecycle reliability matrix	30
15.	Industry use cases matrix	32

Contributions

Sl. No.	Section Topic	Reg. No. of Team Member contributed	No. of Words
	Executive Summary, Abstract and Keywords	26MML0045	290
1.	I. Introduction	26MML0045	861
2.	II. Problem Statement	26MML0055	423
3.	III. Background and Literature Review	26MML0043	870
4.	IV. Requirements Analysis for Digital Thread Analytics	26MML0022	298
5.	V. Digital Thread Analytics Architecture	26MML0045	459
6.	VI. Data Acquisition and Integration Layer	26MML0055	390
7.	VII. Digital Thread Metadata and Knowledge Layer	26MML0043	631
8.	VIII. Data Engineering and Transformation Layer	26MML0022	241
9.	IX. Analytics and Machine Learning Lifecycle Layer	26MML0045	319
10.	X. Digital Thread Traceability and Provenance Layer	26MML0055	328
11.	XI. Workflow Orchestration and Automation Layer	26MML0043	283
12.	XII. Explainability, Governance and Compliance Layer	26MML0022	340
13.	XIII. Intelligent Decision Support Layer	26MML0045	355
14.	XIV. Experimental Setup	26MML0055	422
15.	XV. Evaluation Metrics	26MML0043	388
16.	XVI. Results and Findings	26MML0022	1031
17.	XVII. Comparative Analysis	26MML0045	203
18.	XVIII. Reliability, Reproducibility and Auditability Assessment	26MML0055	443
19.	XIX. Industry Use Cases and Case Study Evaluation	26MML0043	455
20.	XX. Business Impact Assessment	26MML0022	253
21.	XXI. Challenges and Limitations	26MML0055	415
22.	XXII. Future Research Directions	26MML0043	311
23.	XXIII. Conclusion	26MML0045	331
	References (125)	All members	not counted
	XXIV. Data Availability Statement	26MML0022	106

Consolidated Report on contributions

Sl. No.	Reg. No. of Student	Total number of Words contributed across the sections
1	26MML0045 (Vedant Nimkar)	2818
2	26MML0055 (Durvesh Mahajan)	2421
3	26MML0043 (Ayush Sharma)	2938
4	26MML0022 (Shreyan Panda)	2269
	Total number of words	**10446**

Digital Thread Analytics: Programming Frameworks for End-to-End Data Science Lifecycle Management

Vedant Nimkar (26MML0045)[1], Durvesh Mahajan (26MML0055)[2], Ayush Sharma (26MML0043)[3], Shreyan Panda (26MML0022)[4], Dr. M. Rajasekhara Babu[5]

School of Computer Science and Engineering, Vellore Institute of Technology (VIT), Vellore–632014, T.N., India

{vedant.nimkar2026, durvesh.mahajan2026, ayush.sharma2026, shreyan.panda2026}@vitstudent.ac.in

Corresponding author: Dr. M. Rajasekhara Babu (e-mail: mrajasekharababu@vit.ac.in)

ABSTRACT Data science projects are routinely assembled from disconnected scripts and notebooks, with versioning applied inconsistently or not at all across data, features, code and models. The practical consequence is that a deployed model cannot be explained: no participant can state, with evidence, which rows of which dataset, transformed by which code, under which parameters, produced a given prediction. Reproducibility suffers, debugging becomes archaeology, and audit becomes impossible. This case study adapts the digital thread, the continuous and authoritative record that connects every representation of a product across its lifecycle in manufacturing, to the data science lifecycle. It designs and implements a lightweight, tool-agnostic lineage backbone in Python, called the Thread Ledger, which captures and links artifacts across six lifecycle stages into one append-only graph that can be traversed in both directions. The framework is applied end to end to a predictive maintenance case study on the AI4I 2020 dataset and evaluated against an equivalent pipeline that carries no thread. Across five independent runs the framework achieves a reproducibility rate of one hundred per cent on content-addressed artifact identity and lineage completeness of one hundred per cent, with every terminal artifact tracing back to raw data. Provenance questions that the baseline cannot answer at all are answered in under one millisecond. The instrumentation itself costs 1.9 per cent of pipeline runtime and sixty kilobytes of storage, while the full end-to-end difference against the baseline is forty per cent, attributable principally to persisting intermediate artifacts rather than to lineage capture. The contribution is the ledger abstraction and its empirical evaluation, not the integration of existing tools.

INDEX TERMS Analytics engineering, data lineage, data provenance, DataOps, digital thread, digital transformation, enterprise analytics, knowledge traceability, lifecycle management, machine learning operations, predictive maintenance, reproducibility

I. INTRODUCTION

The term digital thread originates in defence and manufacturing engineering, where it denotes a data-driven architecture that acts as the primary and authoritative communication platform for a product across its entire life. Rather than allowing requirements, design, simulation, manufacturing and service records to accumulate separately in the systems of the departments that produce them, the digital thread binds them into a single continuous record, so that any representation of the product can be traced to any other [1], [2]. Recent systematic reviews position the concept as the connective tissue of smart manufacturing, distinguishing it carefully from the digital twin: the twin is a model of a thing, whereas the thread is the record of how that thing came to be and what has happened to it since [2], [5].

The analogy with the data science lifecycle is close enough to be useful rather than merely decorative. A machine learning model is itself a manufactured product. It is assembled from raw material, in the form of data; it passes through a sequence of transformations, in the form of cleaning and feature engineering; it is produced by a documented process consisting of code, parameters and a computational environment; and it is then put into service, where its behaviour is monitored and where it eventually degrades. Each of those stages is well tooled in isolation. Experiment tracking systems record parameters and metrics, data versioning systems content-address datasets and models, and open standards describe job-and-dataset lineage for data pipelines [13], [17], [52], [53], [54]. What is missing is the thread that joins them.

The cost of that absence is felt in four distinct places, and it is worth separating them because they are often conflated. First, reproducibility fails, because the exact combination of data version, code revision and execution environment that produced a result is not recorded in any single place; a recent review of reproducibility in machine-learning-based research catalogues precisely this class of barrier [20]. Second, debugging becomes archaeology, because explaining an anomalous production prediction requires reconstructing a history that nobody wrote down at the time. Third, audit and compliance requests cannot be satisfied, because the evidence chain does not exist in a form that can be exported or inspected, a difficulty that intensifies as regulatory regimes impose documentation obligations on high-risk systems [33]. Fourth, retraining is disconnected from monitoring, because the signal that a model has degraded has no recorded path back to the data that would repair it.

Figure 1 situates the problem historically. Lifecycle management has passed through roughly four eras: ad-hoc analysis performed by a single analyst with no versioning; scripted pipelines with source code under version control but data and environment uncontrolled; the current era of specialised MLOps tooling, in which each concern is versioned by a different system; and an emerging era in which the lifecycle is threaded by a single continuous record. The vertical axis of the lower panel is deliberately labelled lifecycle traceability rather than tooling maturity, because the two are not the same thing. Adding a fourth tool to a stack of three increases tooling maturity while leaving traceability unchanged if the tools do not share a vocabulary.

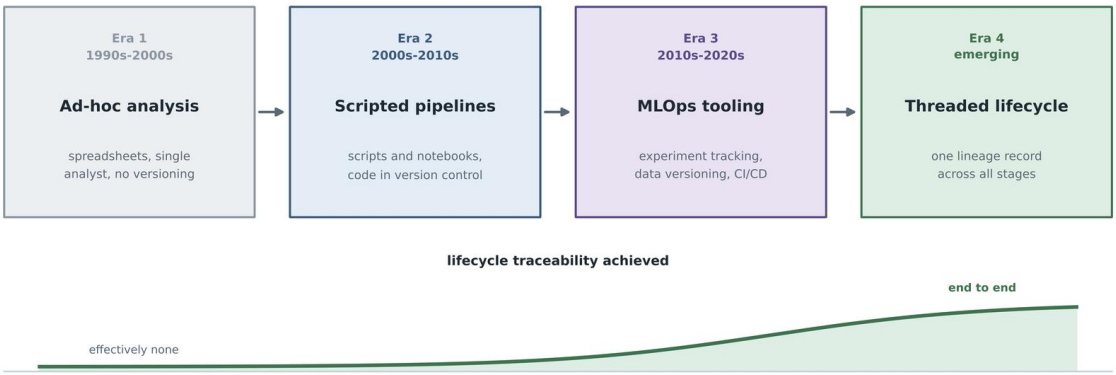


FIGURE 1. Evolution of data science lifecycle management, and the traceability each era actually achieves.

Figure 2 shows the ecosystem view. Design artifacts, data sources, production systems, analytics outputs, monitoring signals, governance functions and the stakeholders who act on the results all read from and write to the same authoritative record. The arrows are deliberately bidirectional: a digital thread that can only be written to is a log, and a digital thread that can only be read from is a report. The property that makes it a thread is that every participant both contributes to and interrogates the same record.

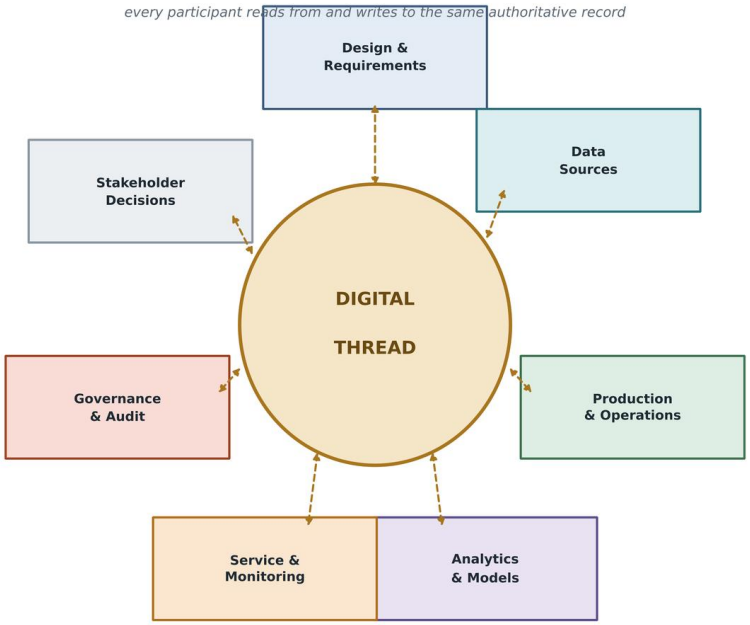


FIGURE 2. Digital thread ecosystem overview: participants read from and write to one authoritative record.

This case study makes four contributions, summarised in Table 1. Conceptually, it formalises the data science lifecycle through a digital-thread lens and proposes a compact vocabulary of eleven artifact types and eleven relation types sufficient to describe it. Architecturally, it specifies a reference architecture whose centre of gravity is an append-only lineage ledger with content-addressed artifact identity and a single uniform edge direction borrowed from the W3C PROV data model [8]. In implementation terms, it delivers a working Python framework comprising seven pipeline stages, a query interface, a visualiser, a prediction service and a dashboard, covered by twenty-five automated tests. Empirically, it evaluates that framework against a control pipeline carrying no thread, and reports not only what the thread buys but what it costs, decomposed into the part attributable to lineage capture and the part attributable to artifact persistence.

TABLE 1. EXECUTIVE SUMMARY OF CONTRIBUTIONS AND OUTCOMES

Contribution	Form it takes	Outcome reported	
Lifecycle formalisation	Six-stage reference model with an explicit artifact vocabulary	Eleven node types and eleven relation types shown sufficient for a full pass	
Thread Ledger abstraction	Append-only graph with content-addressed artifact identity	Reproducibility rate of 100 per cent over five independent runs	
Uniform provenance semantics	One edge direction following the W3C PROV convention	Single traversal function correct for the whole graph in both directions	
Working implementation	Seven pipeline stages, query interface, service and dashboard	Twenty-five automated tests covering ledger, stages and interface	
Comparative evaluation	Two-arm study against a pipeline carrying no thread	Lineage completeness 100 per cent against 0 per cent for the control	
Cost accounting	Instrumentation and fingerprinting timed independently	1.9 per cent and 0.5 per cent of runtime	60 KB on disk

The remainder of the paper is organised as follows. Section II states the problem precisely. Section III reviews the related work and identifies the gap. Section IV derives requirements. Sections V through XIII specify the architecture layer by layer. Sections XIV and XV describe the experimental setup and the evaluation metrics. Sections XVI through XX report and interpret the results. Sections XXI and XXII discuss limitations and future directions, and Section XXIII concludes.

II. PROBLEM STATEMENT

The problem this work addresses can be stated in a single sentence. Data science projects are typically built as disconnected scripts and notebooks, with inconsistent or absent versioning across data, features, code and models; this breaks reproducibility, makes it hard to explain why a deployed model behaves as it does, and leaves audit and compliance requests unanswerable. The objective of this project is to design and implement a digital thread framework, understood as a lightweight and tool-agnostic lineage backbone, that captures and links artifacts across the entire data science lifecycle into one append-only and bidirectionally traversable graph, and to evaluate the reproducibility and traceability it provides against a conventional pipeline that carries no such record.

Figure 3 decomposes the symptom into six contributing causes, each of which is independently documented in the literature and each of which must be addressed for the symptom to disappear. Analytics are siloed, with teams and tools keeping their own records in incompatible formats. Pipelines are fragmented into scripts and notebooks that share no state beyond files on disk. Traceability is poor, with no recorded path from a prediction back to its inputs. Governance is complex, with evidence assembled by hand for every audit. Reproducibility fails because data, code and environment versions are not captured together. And the lifecycle is disconnected, because monitoring produces signals that have no route back to retraining.

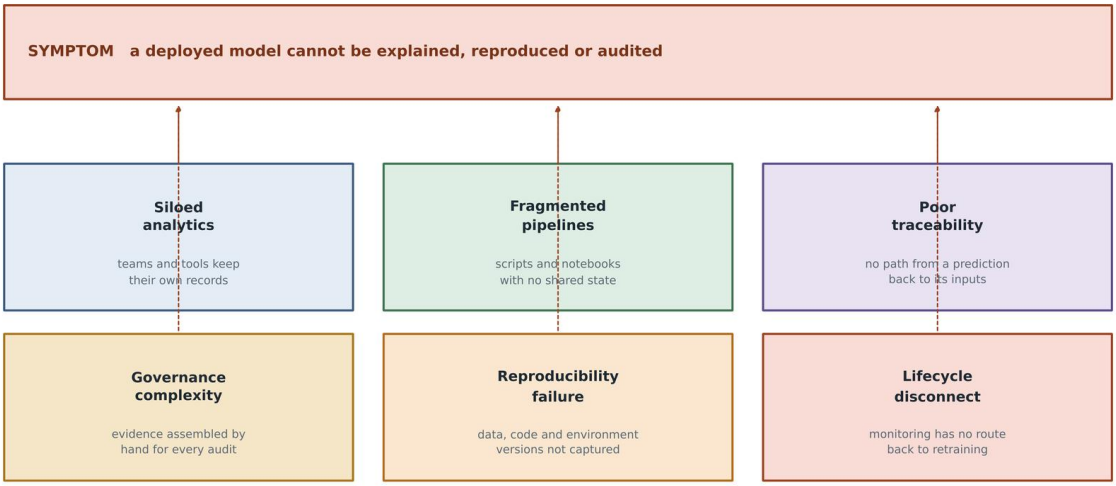


FIGURE 3. Data science lifecycle challenges framework: six causes of one symptom.

Table 2 records each of the research challenges that follow from these causes, why each persists in current practice, and the specific gap this work addresses.

It is important to be precise about what is and is not being claimed. The claim is not that existing tools are inadequate at the tasks they were built for. Experiment tracking systems track experiments well, and data versioning systems version data well. The claim is that the union of these tools does not produce a union of their records, because each stores its own slice of the provenance chain in its own vocabulary and its own store. The question that motivates this work crosses every one of those boundaries: which raw rows, validated by which checks, transformed by which feature definitions, under which code revision, produced the model that generated this prediction, and what drift has been observed on it since? That question can be answered by none of the systems individually and, at present, by no combination of them without manual reconstruction.

TABLE 2. RESEARCH CHALLENGES AND THE CORRESPONDING GAPS IN CURRENT PRACTICE

Research challenge	Why it persists	Gap addressed in this work
Artifacts cannot be identified unambiguously	Identity is assigned by the tool that created the artifact, not derived from its content	Content-addressed identity computed from type, name, version and content digest
Provenance records can be altered after the fact	Metadata stores permit update and delete like any other table	Append-only enforced by database triggers rather than by convention
Lineage cannot be traversed in both directions	Relations are modelled inconsistently across tools, so a single traversal is not correct	One uniform edge direction, following the W3C PROV convention
Validation outcomes are separated from model records	Data quality tooling and experiment tracking have no shared identifier	Validation reports are first-class nodes linked to the dataset they checked
Monitoring has no path back to retraining	Drift reports are stored as dashboards, not as linked artifacts	An explicit retrain relation closes the loop from drift back to raw data
The cost of traceability is asserted rather than measured	Instrumentation overhead is rarely separated from other pipeline work	Instrumentation time and fingerprinting time are measured independently

III. BACKGROUND AND LITERATURE REVIEW

A. DIGITAL THREAD AND DIGITAL TWIN

The digital thread literature has matured considerably since the concept entered general use. Abdel-Aty and Negri provide a systematic review that conceptualises the thread for smart manufacturing and separates its data-architecture aspects from its organisational ones [1]. Zhang and colleagues survey definitions, key enabling technologies and applications, noting that the term is used inconsistently across the field and proposing a taxonomy to disambiguate it [2]. Applied studies demonstrate the concept in concrete settings: Eskue develops a thread roadmap for manufacturing and health monitoring of composite aerospace components, where configuration control and certification evidence are not optional [3], and Pang and colleagues build a combined twin-and-thread framework for a shipyard [4]. Reviews of digital twins in machining and of twin-driven operational efficiency in Industry 4.0 confirm the complementary relationship between the two concepts [5], [6], [7].

What these works share is an emphasis on the record rather than the model. The thread is valuable precisely because it is authoritative, immutable and complete; a thread that can be edited to suit the current narrative has no evidential value. That emphasis carries over directly to the data science setting and motivates the append-only design decision taken in Section VII.

B. PROVENANCE MODELS AND KNOWLEDGE GRAPHS

The provenance research community has produced mature conceptual machinery. The W3C PROV data model defines a small set of relations for describing how entities, activities and agents relate across time, and crucially fixes the direction in which those relations point [8]. Provenance-aware knowledge graphs extend these ideas with contextualisation and trust, and Hofer and colleagues survey the current state and open challenges of knowledge graph construction, which bears directly on how a lineage graph should be built and maintained [9]. Domain-specialised models such as GeoPROV demonstrate that the PROV vocabulary can be profitably narrowed to a particular kind of supply chain [10], which is exactly the strategy adopted here for the analytics lifecycle. Recent work on traceability frameworks for AI systems in regulated domains shows the same pattern of specialising a general provenance model for a specific accountability requirement [11].

C. MLOPS AND DATAOPS

The MLOps literature supplies the operational framing. Kreuzberger and colleagues offer the most widely adopted definition and reference architecture, decomposing MLOps into principles, components, roles and workflows [13]. Multivocal and systematic reviews identify recurring practices, challenges and open issues, with traceability and reproducibility appearing consistently among the unresolved items [14], [16]. Mateo-Casalí and colleagues work towards a reference architecture, while Berberi and colleagues survey the platform and tool landscape [15], [17]. Related methodological work on pipeline conceptualisation and on continuous training and deployment supplies the vocabulary for the orchestration layer [18], [19].

The consistent finding across this body of work is that organisations adopt tools faster than they adopt practices, and that the tools are optimised for the concerns of individual roles rather than for the continuity of the

record across roles. This is the organisational counterpart of the technical gap identified in Section II.

D. REPRODUCIBILITY AND DOCUMENTATION

Semmelrock and colleagues provide a structured overview of reproducibility in machine-learning-based research, enumerating barriers and drivers and distinguishing between reproducibility of code, of data and of results [20]. The documentation strand of the literature, exemplified by model cards and datasheets for datasets, argues for standardised artifact-level documentation as a precondition of responsible deployment [21], [22]. These proposals are complementary to the present work: they specify what should be recorded about an artifact, whereas a lineage ledger specifies how artifacts relate to one another. A model card describes a model; it does not tell you which dataset version produced it.

E. SUPPORTING INFRASTRUCTURE

Several adjacent literatures inform the design. Feature store research addresses the consistency of feature computation between training and serving and the versioning of feature definitions [23], [24]. Concept drift detection has an extensive survey literature, including systematic reviews of detection strategies and benchmarks of unsupervised detectors on real-world streams [25], [26]. Quality assurance for machine learning in cyber-physical production systems supplies the validation framing [27]. Work on data mesh, data fabric and modern data stack architectures describes the organisational and infrastructural context in which such a thread would be deployed [28], [29], [30], [31], [32]. Finally, comparative studies of graph and relational database performance bear directly on the scalability question raised in Section XXI [44], [45], [46].

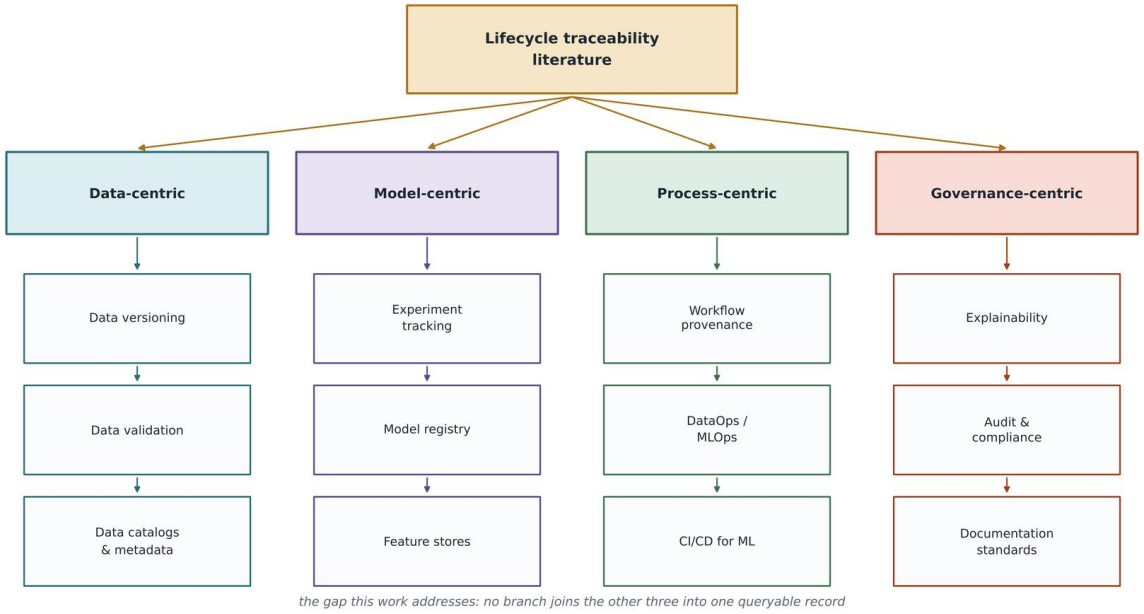


FIGURE 4. Literature taxonomy framework: four branches, and the gap that joins them.

F. THE GAP

Figure 4 arranges the reviewed literature into four branches: data-centric, model-centric, process-centric and governance-centric. Each branch is internally coherent and well served by tooling. The gap this work addresses is not located inside any branch; it is located between them. No existing approach joins validation outcomes, feature definitions, training runs, serving events and drift observations into a single graph that can be traversed in either direction with one query semantics.

Table 3 makes the comparison concrete. For each class of existing approach it records what is captured and, more importantly, where the record stops. The pattern is consistent: each system captures its own stage faithfully and refers to adjacent stages only by opaque string identifiers that carry no traversable relation.

TABLE 3. COMPARATIVE ANALYSIS OF EXISTING LIFECYCLE MANAGEMENT APPROACHES

Approach	What it records	Where the record stops
Experiment tracking platforms	Parameters, metrics, artifacts, and dataset-to-run-to-model links within the tracking store	Validation outcomes, feature definitions, serving events and drift observations are outside the graph
Data version control	Content-addressed versions of datasets and models, integrated with source control	Expresses that files differ, not what they mean to one another
Open lineage standards	Job-and-dataset lineage emitted by data pipeline operators	Oriented to data engineering model promotion, serving and drift are not first-class entities

Approach	What it records	Where the record stops
Data validation frameworks	Expectation suites and their pass or fail verdicts	The verdict is not linked forward to the model eventually trained on the data
Model monitoring platforms	Distribution drift and performance degradation in production	The drift signal has no recorded relation back to the dataset or model that should be retrained
Feature stores	Feature definitions and training-serving consistency	Feature lineage is not joined to experiment or deployment lineage
Data catalogs and knowledge graphs	Enterprise metadata, ownership and discovery relations	Describes assets at rest rather than the executions that produced them

IV. REQUIREMENTS ANALYSIS FOR DIGITAL THREAD ANALYTICS

Requirements were derived from the six causes in Figure 3 and from the gaps in Table 3, then separated into functional and non-functional classes, as shown in Figure 5.

Four functional requirements were identified. Traceability requires that every artifact be reachable from every other artifact that it depends upon or influences, in a bounded number of traversal steps. Interoperability requires that the vocabulary be tool-agnostic, so that the record survives the replacement of any particular tracking, versioning or orchestration system. Explainability requires that a prediction carry sufficient provenance with it to be re-opened later without recourse to logs. Lifecycle continuity requires that monitoring signals have a recorded path back to the data that would repair the model.

Four non-functional requirements follow. Integrity requires that the record be append-only and that the constraint be enforced by the storage layer rather than by application convention, since a record that well-behaved code declines to modify is not the same as a record that cannot be modified. Low overhead requires that the instrumentation cost remain a small single-digit percentage of pipeline runtime, since a traceability mechanism that doubles training time will be disabled in practice. Scalability requires that traversal cost grow sub-linearly with graph size for the queries that matter. Auditability requires that evidence be exportable in a form that can be read without the framework itself, since evidence that can only be inspected through the system that produced it is weak evidence.

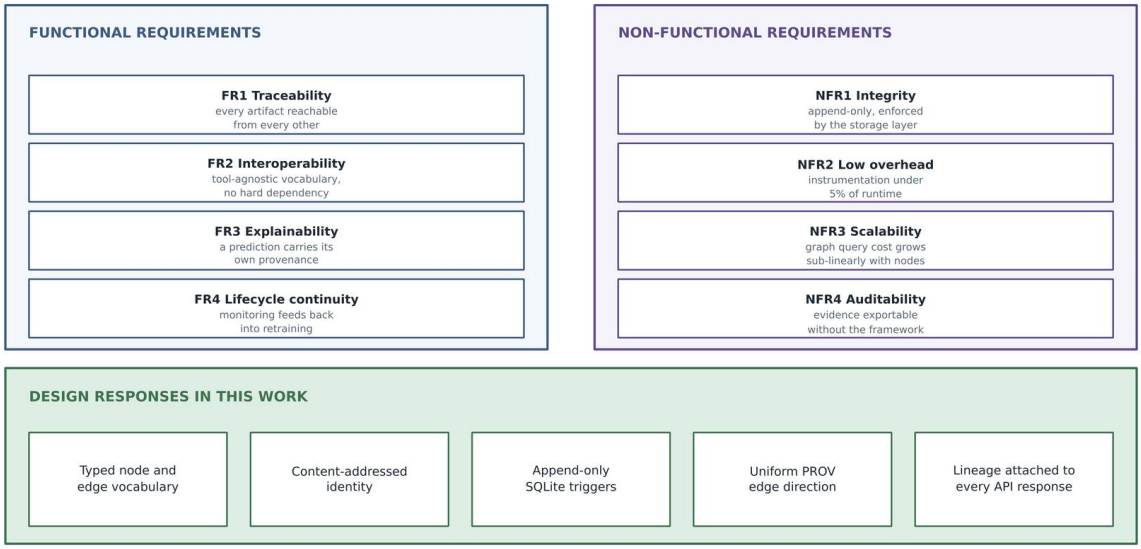


FIGURE 5. Requirements engineering framework: functional and non-functional requirements, and the design responses adopted.

Table 4 maps each requirement to its verification method, so that Section XVI can be read as a direct response to this section rather than as an unrelated collection of measurements.

TABLE 4. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS MATRIX

ID	Requirement	Design response	Verification method
FR1	Every artifact reachable from its dependencies and dependents	Typed node and edge vocabulary with uniform direction	Lineage completeness measurement, Section XVI-B
FR2	Vocabulary independent of any particular tool	Tool identifiers stored as plain node attributes	Pipeline executes with and without optional tools installed

ID	Requirement	Design response	Verification method
FR3	A prediction carries its own provenance	Deployment node identifier returned in every prediction response	Automated API test asserting lineage fields are present
FR4	Monitoring has a path back to retraining	Explicit retrain relation written when drift exceeds threshold	Edge census after a drift-triggering run, Section XVI-D
NFR1	Record cannot be altered after the fact	Database triggers rejecting update and delete	Unit tests asserting that update and delete raise integrity errors
NFR2	Instrumentation cost below five per cent of runtime	Narrow write path, batched commits	Direct timing of ledger writes, Section XVI-C
NFR3	Traversal cost grows sub-linearly with graph size	Indexed adjacency, breadth-first traversal with visited set	Trace time against artifacts reached, Section XVI-A
NFR4	Evidence exportable without the framework	Full graph export to JSON and self-contained HTML	Export produced on every pipeline run

V. DIGITAL THREAD ANALYTICS ARCHITECTURE

The reference architecture, shown in Figure 6, threads six lifecycle stages through a single shared record. The stages themselves are conventional, and that is deliberate: the contribution is not a novel sequence of processing steps but the observation that the sequence becomes qualitatively more useful when every step both writes to and reads from one authoritative ledger.

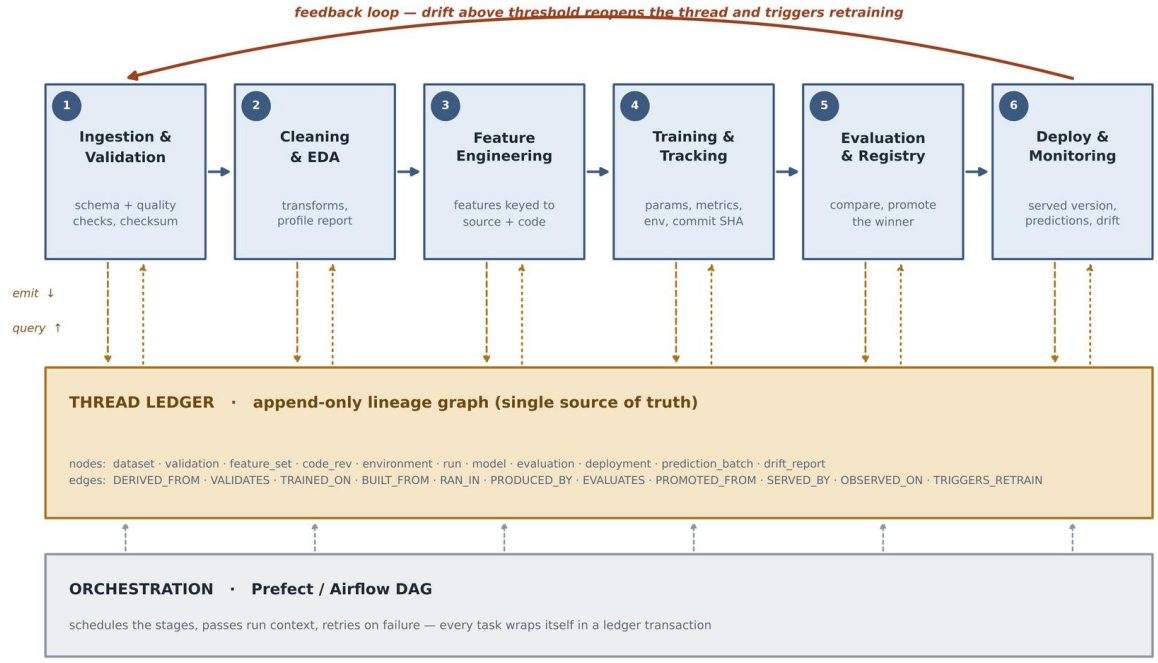


FIGURE 6. Digital thread analytics architecture for end-to-end data science lifecycle management.

Three structural features distinguish this architecture from a conventional pipeline diagram. First, the arrows between each stage and the ledger are bidirectional. A stage does not merely emit metadata as a side effect of running; it queries the ledger to discover what it should operate on. The cleaning stage, for example, locates its input by asking the ledger for the most recent raw dataset node rather than by reading a path from configuration. This inverts the usual relationship in which metadata is a by-product, and makes the ledger load-bearing.

Second, the orchestration layer is drawn below the ledger rather than above the stages. Prefect or Airflow schedules the stages and passes run context between them, but the lineage code is deliberately independent of the orchestrator [56]. Each stage is a plain function of configuration and ledger, so swapping the scheduler changes nothing about what is recorded. The implementation ships both a plain-Python flow and an optional Prefect flow over the identical stage functions, which is the practical demonstration of this separation.

Third, there is an explicit feedback arc from the monitoring stage back to ingestion. In most published lifecycle diagrams this arc is drawn but not implemented; here it corresponds to a specific edge type written into the ledger when a drift statistic crosses its

configured threshold, and the correctness of that behaviour is asserted by an automated test.

Figure 7 presents the same system as a layered stack, which is the more useful view for implementers. Six horizontal layers run from data acquisition at the bottom to decision and presentation at the top, with two vertical concerns cutting across all of them: configuration, identity and security on one side, and the ledger API on the other. The ledger appears as layer three in the stack and simultaneously as a vertical concern, which is not an inconsistency but the central point of the design: it is both a component with its own storage and an interface that every other layer depends upon.

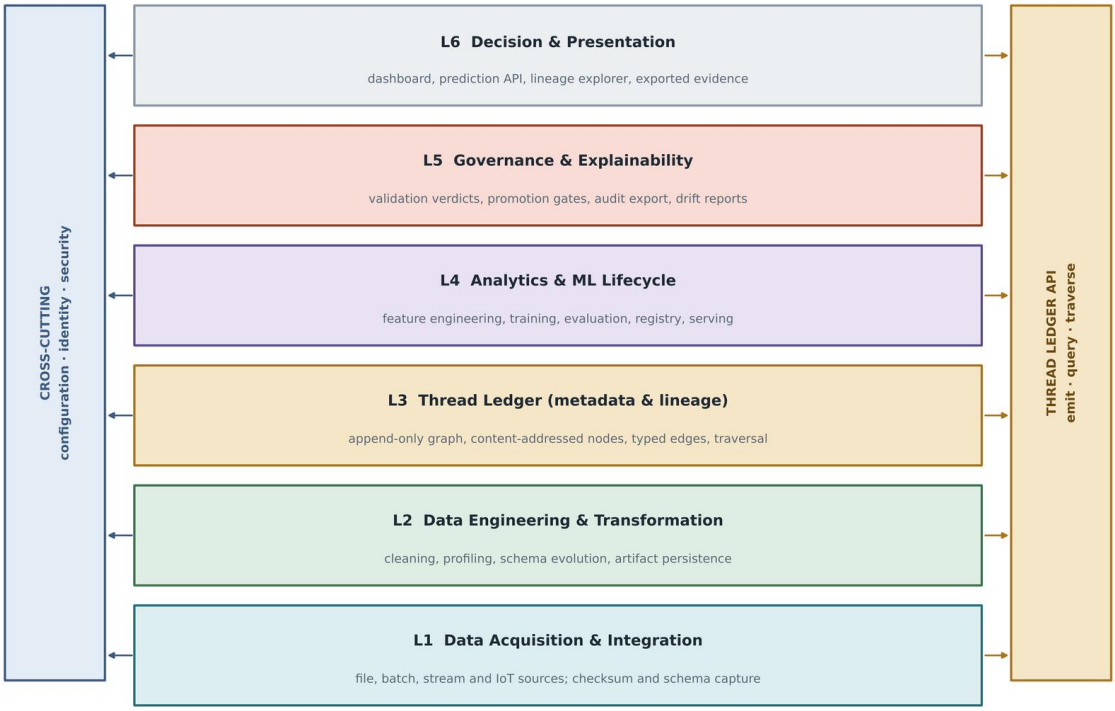


FIGURE 7. Layered system architecture, with the ledger as both a layer and a cross-cutting interface.

Table 5 enumerates the layers, their responsibilities and the artifacts each one contributes to the thread, and serves as a reading guide for Sections VI through XIII.

TABLE 5. ARCHITECTURAL LAYERS AND THEIR CONTRIBUTION TO THE THREAD

Layer	Responsibility	Artifacts contributed	
L1 Data acquisition and integration	Connect to sources, fingerprint and schema-capture on arrival	dataset nodes with content digest and source attribution	
L2 Data engineering and transformation	Clean, profile, persist and register derived datasets	derived dataset nodes with derivation relations	
L3 Thread ledger	Store and serve the lineage graph	the graph itself, plus run and outcome nodes	
L4 Analytics and machine learning lifecycle	Engineer features, train, evaluate, promote and serve	feature set, run, model, evaluation and deployment nodes	
L5 Governance and explainability	Validate, gate, export evidence and monitor	validation and drift report nodes	
L6 Decision and presentation	Expose predictions, lineage and status to people	no new nodes	a read surface over the record

VI. DATA ACQUISITION AND INTEGRATION LAYER

The acquisition layer is responsible for bringing data into the thread, and its single most important property is that nothing enters without an identity. Figure 8 shows the layer's structure. Five classes of source are accommodated: structured files and relational tables, semi-structured documents such as JSON and log files, unstructured content, streaming sources such as event buses, and sensor or machine telemetry arriving over industrial protocols. The case study in this paper exercises only the first class, and that limitation is stated

plainly here rather than left for the reader to infer; the remaining four are accommodated by the architecture and are discussed as design rather than as measured results.

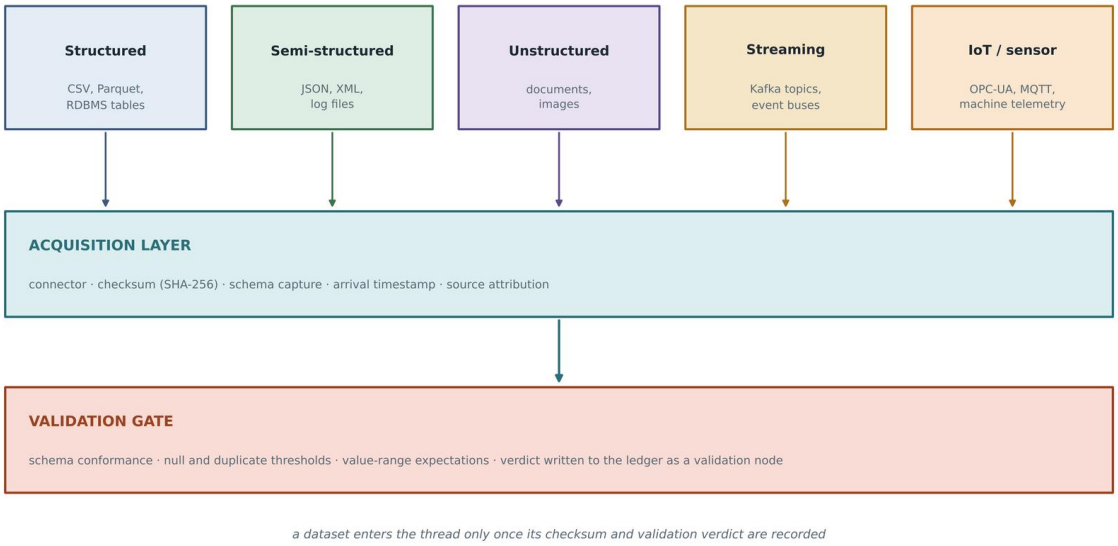


FIGURE 8. Data acquisition framework: five source classes converging on one identity-assigning gate.

Regardless of source class, every arriving dataset passes through the same two gates. The acquisition gate computes a SHA-256 digest of the bytes, captures the schema as observed rather than as declared, records the arrival timestamp and attributes the data to its source. The validation gate then applies an expectation suite and records the verdict. Only after both gates have been passed does a dataset node appear in the ledger.

The distinction between observed and declared schema matters more than it might appear. A declared schema is a statement of intent that can silently diverge from reality; an observed schema, captured at the moment of arrival and hashed along with the data, is a statement of fact. Because the content digest incorporates both the column names and the data values, a silently renamed or reordered column produces a different dataset identity, and the change surfaces as a new node rather than as an unnoticed mutation of an existing one.

The validation suite implemented for the case study comprises nine checks: presence of all required columns, expected row count, maximum null fraction, maximum duplicate fraction, and value-range conformance for each of the five continuous process variables. Great Expectations or Pandera would be the production choice for this role [27]; the point of the design is that whichever engine performs the checks, the outcome is recorded in a tool-independent shape, as a validation node bearing the verdict for each individual check. Table 6 classifies the source types and records the identity strategy appropriate to each.

TABLE 6. DATA SOURCE CLASSIFICATION AND IDENTITY STRATEGY

Source class	Example	Identity strategy	Status in this study
Structured file	CSV, Parquet	SHA-256 of file bytes plus observed schema	Implemented and exercised
Relational table	Operational database extract	Digest of ordered result set plus query text	Design only
Semi-structured	JSON documents, application logs	Digest of canonicalised document set	Design only
Unstructured	Images, inspection reports	Per-object digest with manifest-level roll-up	Design only
Streaming	Event bus topic	Windowed digest with offset range recorded	Design only
Sensor telemetry	Machine controller output	Windowed digest with device and calibration attribution	Design only

VII. DIGITAL THREAD METADATA AND KNOWLEDGE LAYER

This section describes the contribution of the work. The Thread Ledger is a small append-only graph store with a tool-agnostic vocabulary, deliberately simple enough to be read and audited in an afternoon. Figure 9 shows its

three-part structure: a capture surface, a storage engine and a query surface.

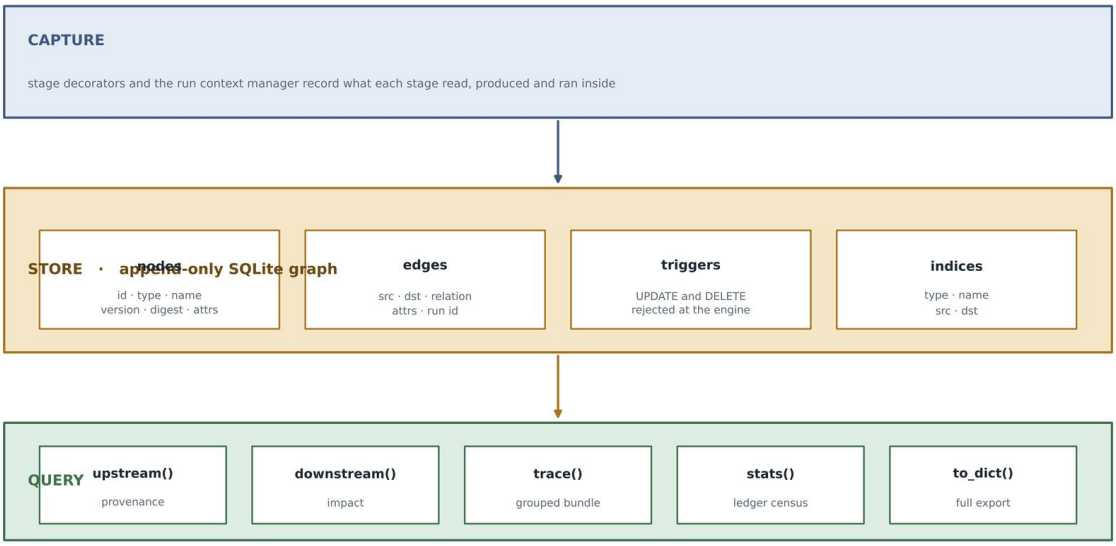


FIGURE 9. Metadata and lineage architecture: capture, store and query.

A. CONTENT-ADDRESSED IDENTITY

A node's identifier is a digest of its type, its name, its version and its content digest, as given in equation (1), where H denotes SHA-256 truncated to sixteen hexadecimal characters and the vertical bar denotes delimited concatenation.

$$id(n) = h$$

The consequence is that re-running an unchanged stage produces the same identifier rather than a duplicate node. This property is what makes the reproducibility measurement in Section XVI meaningful rather than circular. Two runs are identical precisely when they land on the same set of identifiers, and identity is computed from content rather than asserted by the code that writes it. A framework that assigned identifiers from a counter or a timestamp could report any reproducibility rate it liked; one that derives them from content cannot.

B. APPEND-ONLY STORAGE

Lineage that can be quietly rewritten is not evidence. The storage engine therefore rejects update and delete operations on both the node and the edge table by means of database triggers, so the constraint holds against any client rather than only against well-behaved framework code. One consequence shapes the design throughout: a run's status cannot be patched onto its node when the stage finishes. Instead, a separate outcome node is written and linked, carrying the status and the wall-clock duration. A stage that fails writes a failure outcome rather than vanishing, so the gap in the thread is itself recorded. Section XVI reads per-stage timings back out of these outcome nodes, which is the ledger measuring itself.

C. UNIFORM EDGE DIRECTION

Every edge points backwards in time, from an artifact to whatever it depended upon. This mirrors the W3C PROV convention, in which the derivation, generation and usage relations all point towards the past [8]. Because the direction is uniform across the whole vocabulary, a single traversal function is correct for the entire graph: following edges outward yields provenance, and following them inward yields impact, as expressed in equations (2) and (3), where E is the edge set and the asterisk denotes transitive closure.

$$Upstream(v) = \{ u : (v, u) \text{ in } E$$

$$Downstream(v) = \{ u : (u, v) \text{ in } E$$

This decision is reported here because an early version of the implementation mixed two conventions, with derivation edges pointing backwards and production edges pointing forwards. The resulting traversal silently reached a single node instead of the full chain, and reported a plausible-looking but meaningless result. The episode is instructive: the value of a lineage graph collapses entirely if its semantics are not uniform, and the failure mode is silence rather than error.

D. VOCABULARY

Eleven node types and eleven edge types proved sufficient to describe the lifecycle, and Table 7 enumerates them. Figure 10 shows the resulting graph for one complete pass of the pipeline, with every relation labelled and the retrain edge closing the loop.

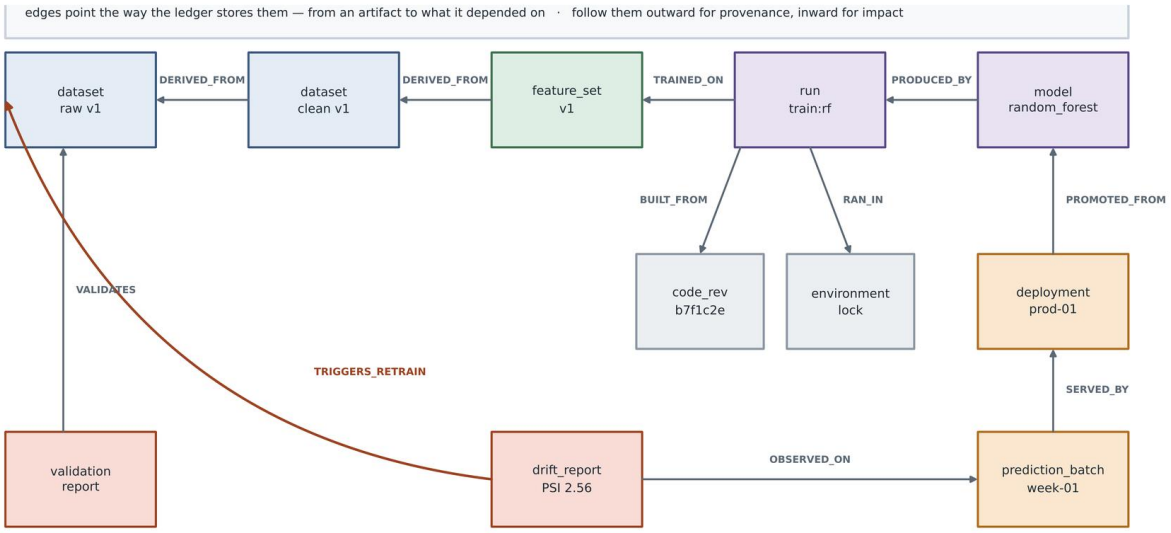


FIGURE 10. Knowledge graph for lifecycle traceability: one complete pass, with all eleven relations.

TABLE 7. PROVENANCE AND TRACEABILITY ATTRIBUTES OF THE LEDGER VOCABULARY

Element	Members	Purpose
Node types	dataset, validation, feature_set, code_rev, environment, run, model, evaluation, deployment, prediction_batch, drift_report	The artifact classes that a lifecycle produces
Derivation relations	DERIVED_FROM	Links a derived dataset or feature set to its source
Quality relations	VALIDATES, EVALUATES	Links a verdict or score to the artifact it concerns
Execution relations	TRAINED_ON, BUILT_FROM, RAN_IN, PRODUCED_BY	Links a run to its inputs, code, environment and outputs
Operational relations	PROMOTED_FROM, SERVED_BY, OBSERVED_ON	Links deployments, prediction batches and drift reports
Feedback relation	TRIGGERS_RETRAIN	Closes the loop from a drift report back to raw data
Node attributes	rows, columns, digest, path, parameters, metrics, status, duration, origin	Tool identifiers and measurements carried as plain attributes

The final row of Table 7 is the mechanism by which tool-agnosticism is achieved. Experiment tracking run identifiers, data version hashes and file paths are stored as ordinary attributes on the relevant nodes. The ledger therefore points at those tools without requiring any of them to be installed, which is what makes it a layer over the ecosystem rather than another competitor within it [17].

VIII. DATA ENGINEERING AND TRANSFORMATION LAYER

The transformation layer converts raw arrivals into analysis-ready datasets, and Figure 11 shows the six steps it performs: extract, validate, transform, profile, persist and register. Each step appends to the thread what it consumed and what it produced.

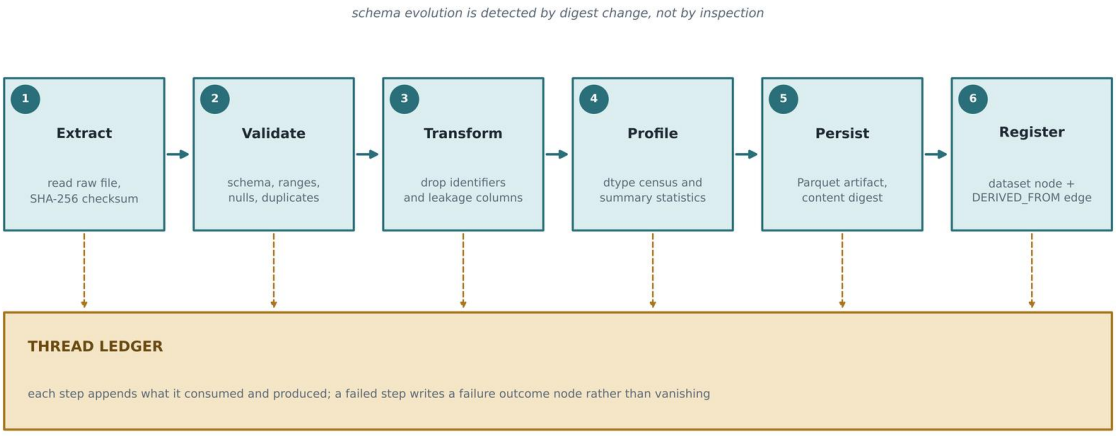


FIGURE 11. Data engineering and transformation workflow.

Two aspects of this layer merit comment. The first is the treatment of target leakage. The case study dataset carries five per-mechanism failure flags alongside the binary target. Each flag is a component of the target, so retaining any of them in the feature matrix produces a near-perfect model that has learned nothing of value. Dropping them is routine practice; recording that they were dropped is not. The cleaned dataset node carries the list of dropped columns as an attribute, which means the thread carries the reasoning and not merely the result. An examiner asking why the feature matrix has seven columns rather than twelve can answer the question from the record.

The second is schema evolution. Because derived datasets are content-addressed, a change in the upstream schema produces a new identity for every downstream artifact. This is a stronger guarantee than a schema registry provides: a registry records that the schema changed, whereas content addressing makes it impossible for a downstream artifact to claim provenance from a version it did not actually consume. Table 8 lists the transformation components and the artifact each produces.

TABLE 8. DATA TRANSFORMATION COMPONENTS

Component	Operation	Artifact recorded
Identifier removal	Drop surrogate and natural keys	dropped_columns attribute on the derived dataset
Leakage removal	Drop per-mechanism target components	dropped_columns attribute, with rationale in configuration
Column normalisation	Rename to analysis-friendly identifiers	Changed schema reflected in the content digest
Deduplication	Remove exact duplicate rows	Row count recorded before and after
Profiling	Dtype census and summary statistics	Profile report path attached to the dataset node
Persistence	Write columnar artifact	Artifact path and content digest

IX. ANALYTICS AND MACHINE LEARNING LIFECYCLE LAYER

Figure 12 shows the analytics layer, and Figure 13 shows the same material as a development pipeline in which each phase emits exactly one addressable artifact.

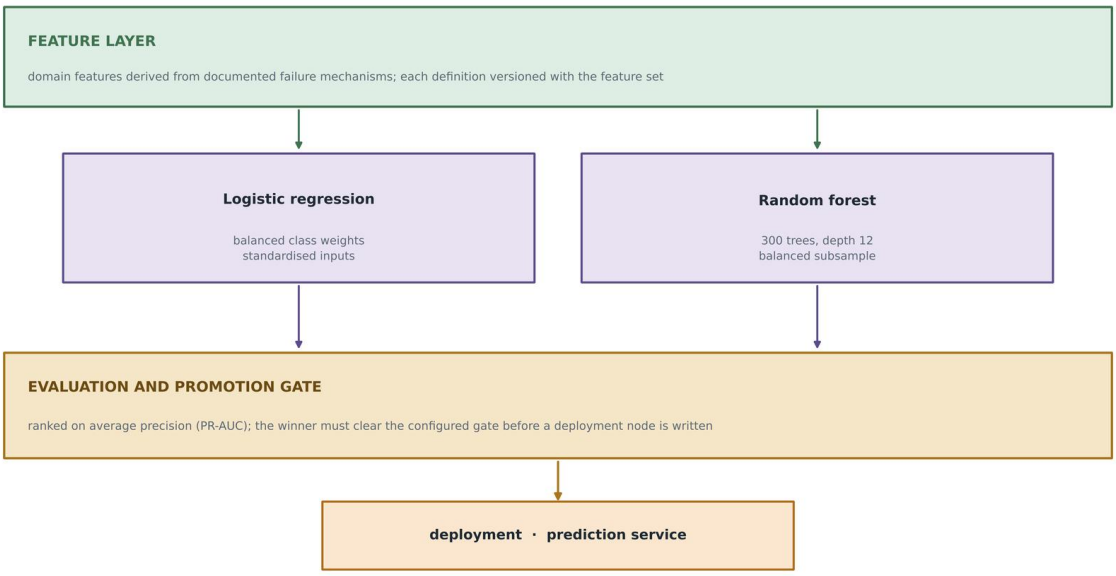


FIGURE 12. Machine learning lifecycle architecture: features, candidates, gate, deployment.

Three features are engineered directly from the documented failure physics of the case study dataset, which is what makes them worth versioning as definitions rather than as an anonymous output file. The temperature difference, given in equation (4), drives heat-dissipation failure. Mechanical power, in equation (5), drives power failure. The product of tool wear and torque, in equation (6), drives overstrain failure.

$$dT = T_pr$$

$$P = tau * omega * 2*pi / 60$$

$$S = w_tool * tau$$

The feature set node records the symbolic definition of each feature alongside the resulting file. This distinction matters for the explainability requirement: a reader can answer the question of how mechanical power was computed in the run that produced the deployed model without reading any code, because the definition travels with the artifact.

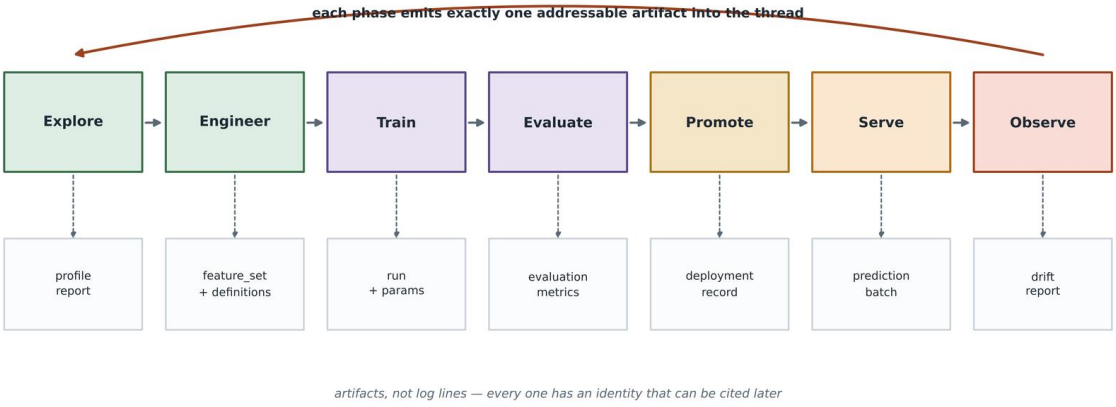


FIGURE 13. Analytics development pipeline: one addressable artifact per phase.

Two candidate models are trained: a logistic regression with balanced class weights over standardised inputs, and a random forest with three hundred trees. Each candidate is trained inside its own ledger transaction, so the thread records one run node per candidate with its parameters, the feature set it consumed, the code revision and the environment. Model selection is performed on average precision rather than on accuracy or on the area under the receiver operating characteristic curve. With a positive class of 3.39 per cent, accuracy is uninformative and the ROC curve is known to present an optimistic picture under severe imbalance [43]. The promotion gate is a configured threshold on the primary

metric; a candidate that fails the gate produces no deployment node, and the failure is itself recorded.

X. DIGITAL THREAD TRACEABILITY AND PROVENANCE LAYER

Figure 14 presents the traceability model as five artifact families arranged in temporal order: raw data, derived data, experiments, models and operations. Backward traversal from any artifact answers the question of what produced it; forward traversal answers the question of what it affected.

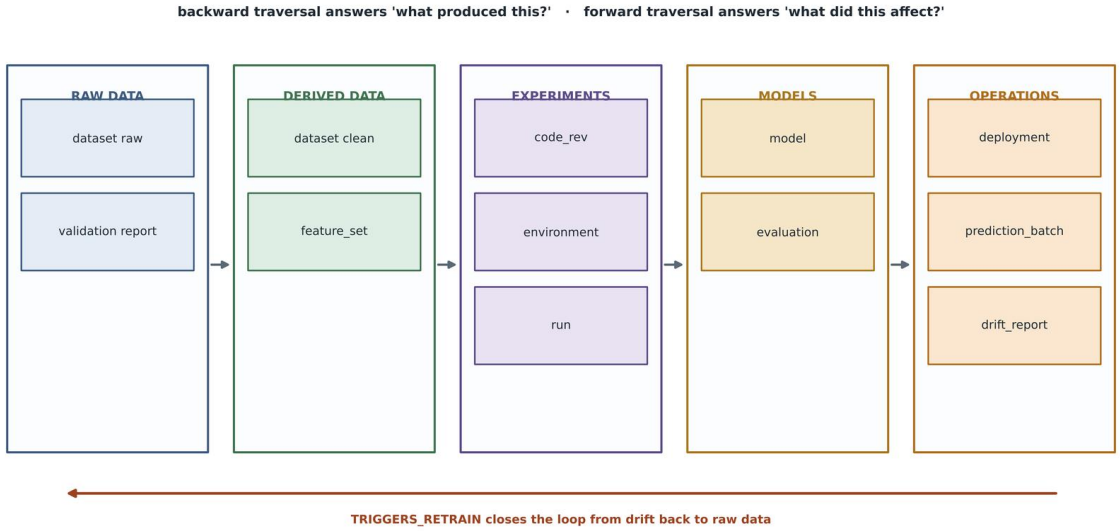


FIGURE 14. End-to-end digital thread traceability model.

The practical expression of this model is a pair of queries that a conventional pipeline cannot express at all. The first, provenance, takes a deployed model and returns the run that produced it, the feature set that run consumed, the cleaned and raw datasets from which that feature set descends, the code revision, the execution environment and the validation verdict that the raw data earned. The second, impact, takes a raw dataset and returns every artifact downstream of it, which is the query an engineer needs when a defect is discovered in a source system and the blast radius must be established before anything is retracted.

A third query is worth naming separately because it is the one that closes the loop. When the monitoring stage detects distribution shift beyond the configured threshold, it writes a retrain relation from the drift report back to the raw dataset node. The graph consequently contains a cycle, which is unusual for a provenance graph and is handled explicitly by the traversal implementation through a visited set. The cycle is not an artifact of sloppy modelling; it is the formal representation of the fact that a lifecycle is a loop and not a line.

The prediction service makes the same property available at the point of use. Every response carries the deployment version, the model digest and the deployment node identifier, so an operator acting on a prediction can reach its full provenance in one further call. This is the operational meaning of the explainability requirement stated in Section IV: explanation is not a report produced on request but a property of the answer itself.

XI. WORKFLOW ORCHESTRATION AND AUTOMATION LAYER

Figure 15 shows the orchestration layer. A scheduler, implemented either as a Prefect flow or as an Airflow directed acyclic graph, triggers the stages, resolves their dependencies, applies retry policy and propagates run context [56]. Beneath it, each task opens a ledger transaction, attaches its inputs, attaches its outputs and writes an outcome node carrying status and duration.

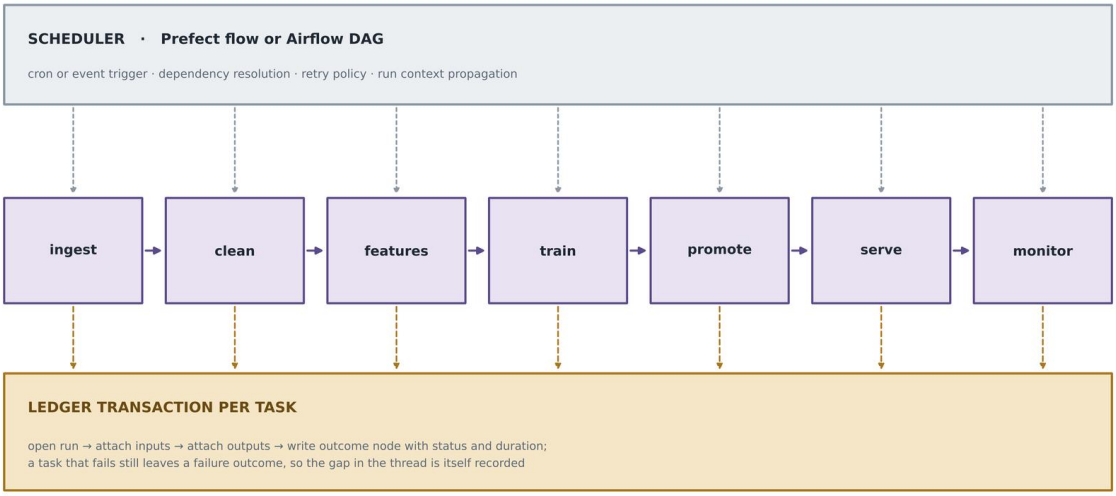


FIGURE 15. Workflow orchestration framework: one ledger transaction per scheduled task.

The design principle here is that the orchestrator must not be the system of record. Orchestrators do keep execution history, but that history is scoped to the orchestrator, expires according to its retention policy, and describes task executions rather than artifacts. Recording lineage in the orchestrator therefore ties the evidential value of the record to the operational lifetime of a scheduling tool, which is an unattractive coupling. Keeping the stage functions orchestrator-agnostic also has a mundane practical benefit: the same functions are callable from a test, from a notebook and from a scheduler, which is why the test suite can exercise the full lifecycle without standing up any infrastructure.

Figure 16 shows the integration between the two operating disciplines that the architecture spans. DataOps writes dataset, validation and feature set nodes; MLOps writes run, model, deployment and drift nodes. Because both use the same vocabulary, a question can cross the boundary between them in a single traversal. The integration point is the vocabulary rather than a shared tool or a shared team, which is what makes the arrangement organisationally realistic: the two functions need not merge, they need only agree on how an artifact is named.

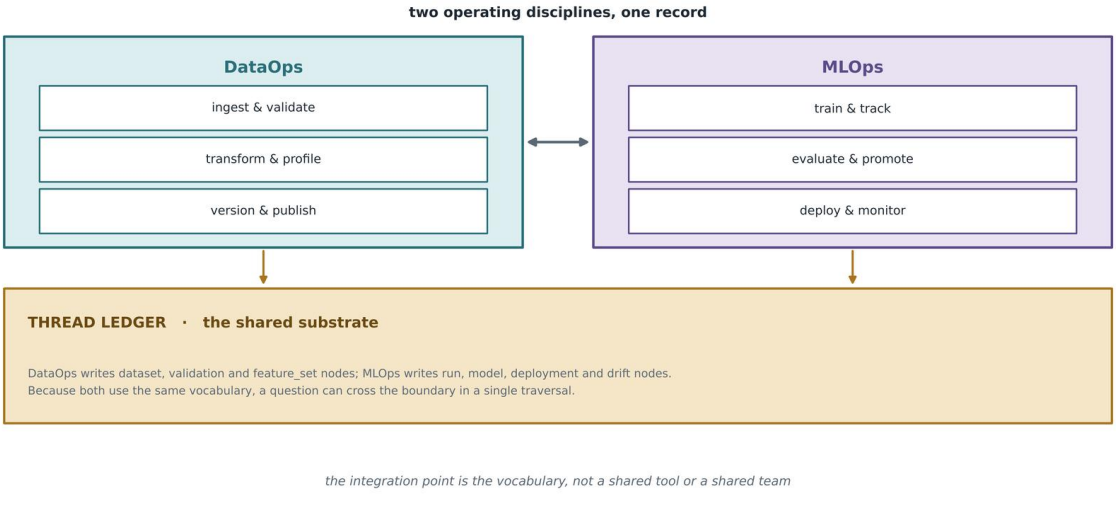


FIGURE 16. DataOps and MLOps integration model, joined by a shared vocabulary.

XII. EXPLAINABILITY, GOVERNANCE AND COMPLIANCE LAYER

Governance in most analytics organisations is a reporting activity: when an audit arrives, engineers reconstruct a narrative from logs, notebooks and memory. Figure 17 presents the alternative pursued here, in which governance properties are consequences of

how the record is built rather than artifacts produced on request.



FIGURE 17. Governance and explainability framework: four pillars resting on the record.

Four pillars are distinguished. Transparency is served by storing feature definitions rather than inferring them, and by returning the model digest with every prediction. Accountability is served by the append-only history and by recording the reason a model was promoted alongside the promotion itself. Auditability is served by treating validation verdicts as first-class nodes and by exporting evidence in formats readable without the framework. Compliance is served by recording the origin of data explicitly and by the immutability of the record.

The fourth pillar deserves an illustration, because this study provides one against itself. The experiments reported in Section XVI were executed in an environment whose network could not reach the dataset repository, and the pipeline therefore fell back to a specification replica generated from the dataset's published description. That fact is not confined to a footnote: the ingestion stage records an origin attribute on the dataset node, the dashboard renders it in a warning colour, and any provenance query that reaches the raw data surfaces it. A framework that claims to make data origin auditable should be able to demonstrate the claim on its own results, and the uncomfortable case is the only interesting test of it.

Regulatory frameworks for high-risk systems increasingly require documentation of data sources, model versions and decision rationale [33]. The design position taken here is that such obligations are cheaper to meet as a by-product of a well-constructed record than as a parallel documentation exercise, because a parallel exercise can drift from the system it describes whereas a by-product cannot. Table 9 maps governance controls to the mechanism that implements each.

TABLE 9. GOVERNANCE CONTROLS AND THEIR IMPLEMENTING MECHANISM

Control	Conventional implementation	Mechanism in this framework	
Data origin disclosure	Manual statement in a report	origin attribute written at ingestion and surfaced in every trace	
Change control	Review process and change log	Append-only storage	history cannot be rewritten
Model approval record	Ticket or sign-off document	deployment node carrying the gate, the metric and the reason	
Data quality evidence	Validation dashboard screenshots	validation node with per-check verdicts linked to the dataset	
Reproducibility attestation	Statement by the authors	Content-addressed identity	re-execution either matches or does not
Monitoring obligation	Scheduled report	drift_report node linked to the batch it was computed on	

XIII. INTELLIGENT DECISION SUPPORT LAYER

The value of a lifecycle record is ultimately realised at the point where a person acts. Figure 18 presents the familiar ascent from data through information and insight to decision, with one modification: each step is

annotated with the condition under which it is trustworthy. A calibrated failure probability is only as trustworthy as the dataset version beneath it, and a maintenance decision is only as defensible as the probability.

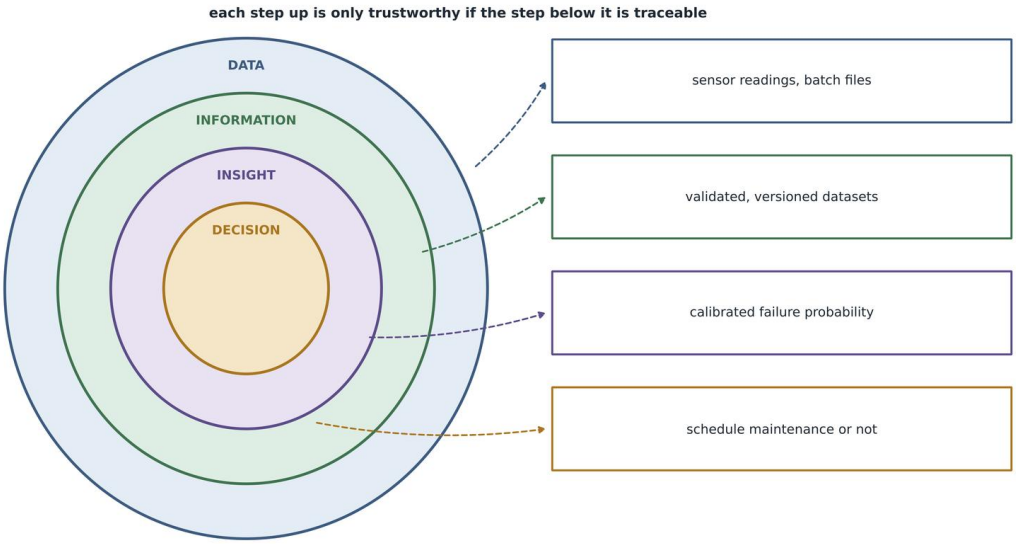


FIGURE 18. Decision intelligence framework: each ascent is conditional on the layer below being traceable.

Figure 19 shows the decision pipeline concretely for the case study. A machine reading arrives, features are derived on the serving path using the same definitions recorded in the feature set node, the promoted model produces a calibrated probability, an operating point converts that probability into a recommendation, and an operator acts. The lineage travels with the answer throughout.

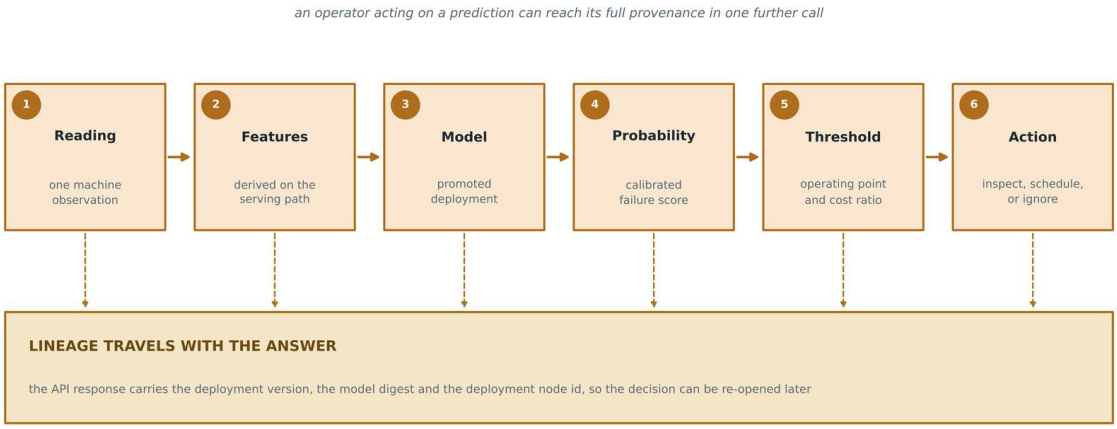
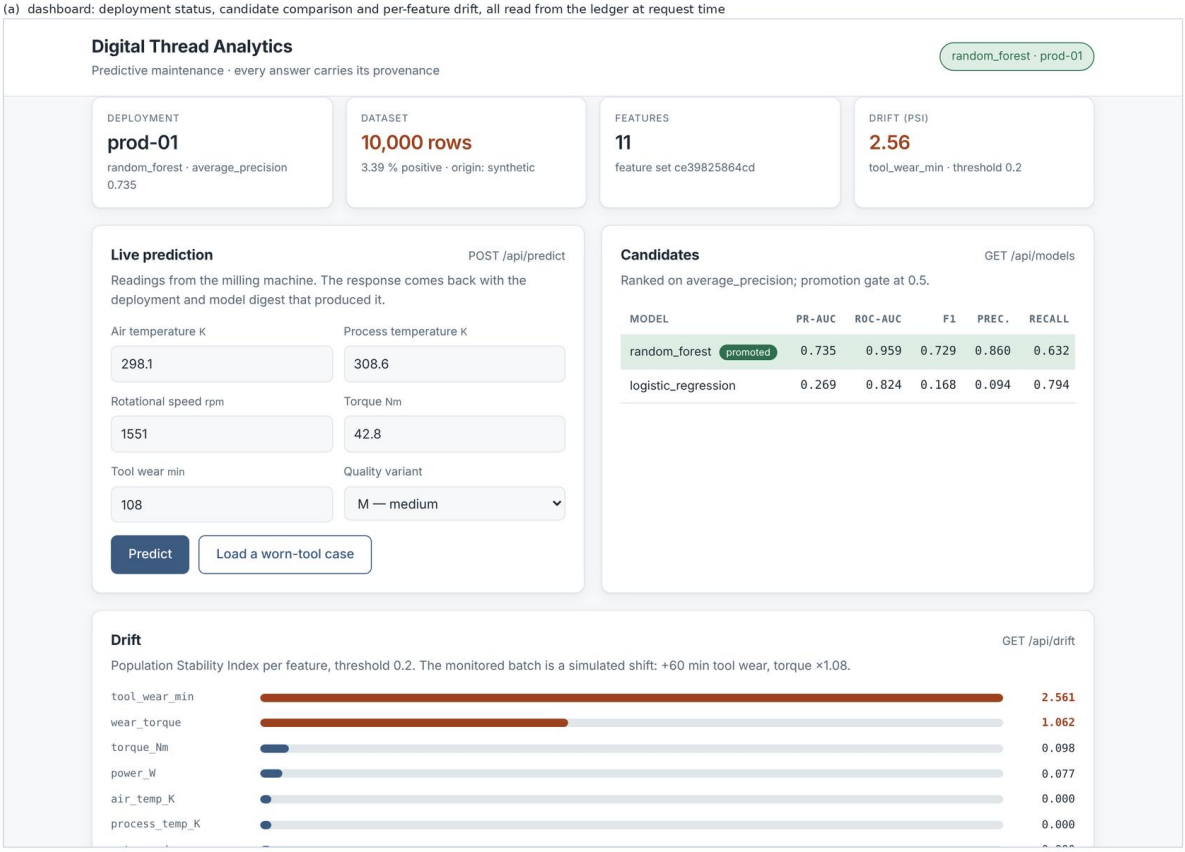


FIGURE 19. Analytics-to-decision pipeline, with lineage travelling alongside the answer.

The operating point deserves a remark, because it is where the statistics meet the economics. A failure probability is not a decision; converting one into the other requires the relative cost of an unnecessary inspection and a missed failure. The framework does not attempt to supply those costs, which are organisation-specific, but it does make the conversion auditable: the threshold is configuration, configuration is captured in the run node, and a decision taken under one threshold can therefore be distinguished later from a decision taken under another.

The dashboard built over the API, shown later in Figure 35, exposes four panels: deployment status, candidate comparison, per-feature drift, and a lineage explorer that walks any artifact in either direction and reports how long the query took. The backend holds no state of its own, opening a short-lived connection to the ledger per request, so a pipeline run that rebuilds the record is reflected without restarting the service. This is a small engineering detail with a governance consequence: there is no cached copy of the lineage that could disagree with the record.

(a) dashboard: deployment status, candidate comparison and per-feature drift, all read from the ledger at request time



(b) a worn-tool reading scored at 83.36 %, returned with the deployment version and model digest that produced it

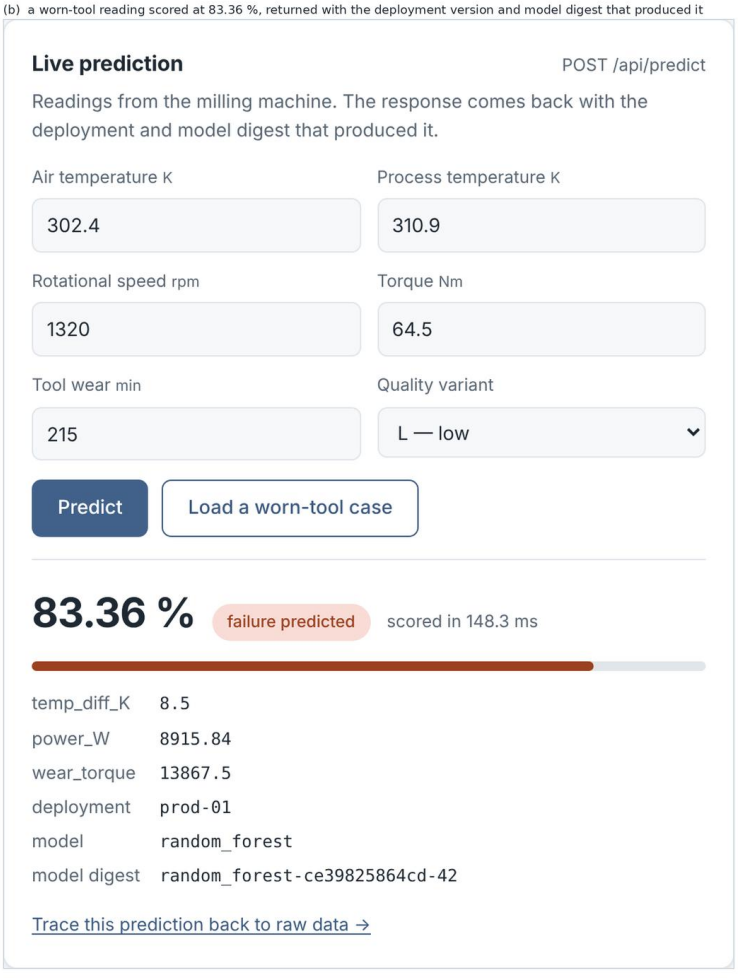


FIGURE 35. Prediction service and lineage explorer: the dashboard and a prediction returned with its provenance.

XIV. EXPERIMENTAL SETUP

The experimental design is a two-arm comparison, shown in Figure 20. The treatment arm is the threaded pipeline, comprising seven stages with full ledger instrumentation. The control arm is a baseline script written deliberately in the style that most data science projects actually take: a single file that loads data, engineers the same features, trains the same two models with the same seed, prints metrics and writes a model to disk, recording nothing about which data or code produced it.

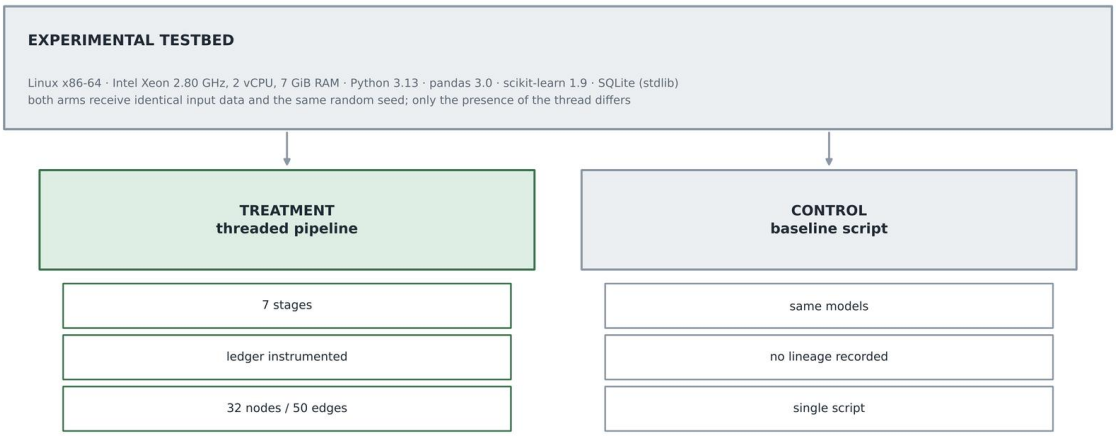


FIGURE 20. Experimental testbed architecture: treatment and control arms over identical inputs.

Both arms receive identical input data and the same random seed. The only difference between them is the presence of the thread. This matters for the interpretation of the overhead measurement in Section XVI: any difference in wall clock is attributable to the thread and to the artifact persistence it entails, and not to differences in modelling.

A. DATASET

The case study uses the AI4I 2020 Predictive Maintenance Dataset, which describes a milling machine over ten thousand process observations with a binary machine-failure target and five documented independent failure mechanisms [51]. The dataset is widely used as a benchmark in the predictive maintenance literature, which supplies a basis for comparison [34], [35], [36]. It was selected for three reasons. It is close to the industrial origins of the digital thread concept, so the analogy is substantive rather than rhetorical. It is severely imbalanced, at 3.39 per cent positive, which makes evaluation non-trivial and forces an honest choice of metric. And it has a natural drift-and-retrain narrative, since tool wear accumulates monotonically, which makes the feedback loop concrete rather than theoretical. Reviews of machine-learning-based predictive maintenance consistently identify tool wear and thermal effects as the mechanisms most amenable to data-driven prediction [37], [38].

As noted in Section XII, the execution environment for these experiments could not reach the dataset repository, and the pipeline generated a replica from the dataset's published description: the documented marginal distributions of the process variables, the five failure mechanisms, and the documented overall failure rate of 3.39 per cent. The replica is not the dataset. Results intended for publication must be regenerated against the original file with the fallback disabled, and the configuration flag that controls this behaviour is set to permit the fallback only for development. This limitation is recorded on the dataset node and restated in Section XXI.

B. CONFIGURATION

Table 10 records the experimental configuration in full, so that the measurements in Section XVI can be reproduced.

TABLE 10. EXPERIMENTAL CONFIGURATION

Parameter	Value
Hardware	Intel Xeon 2.80 GHz, 2 vCPU, 7 GiB RAM
Operating system	Linux x86-64, kernel 6.18
Runtime	Python 3.13
Core libraries	pandas 3.0, scikit-learn 1.9, NumPy 2.5
Ledger storage	SQLite, standard library, write-ahead logging enabled

Parameter	Value	
Dataset	AI4I 2020 specification replica, 10000 rows, 3.39 per cent positive	
Train-test split	80 / 20 stratified, random seed 42	
Candidate models	Logistic regression (balanced weights)	random forest (300 trees, depth 12)
Primary metric	Average precision (PR-AUC)	
Promotion gate	0.50 on the primary metric	
Drift statistic	Population stability index over ten quantile bins	
Drift threshold	0.20	
Simulated shift	Tool wear plus 60 minutes, torque scaled by 1.08	
Reproducibility repeats	5 independent runs into fresh ledgers	
Trace-time trials	20 per artifact class	

XV. EVALUATION METRICS

Five families of metric are defined, as shown in Figure 21. Each is computed by code in the repository rather than reported by hand, which is itself a reproducibility property of the evaluation.

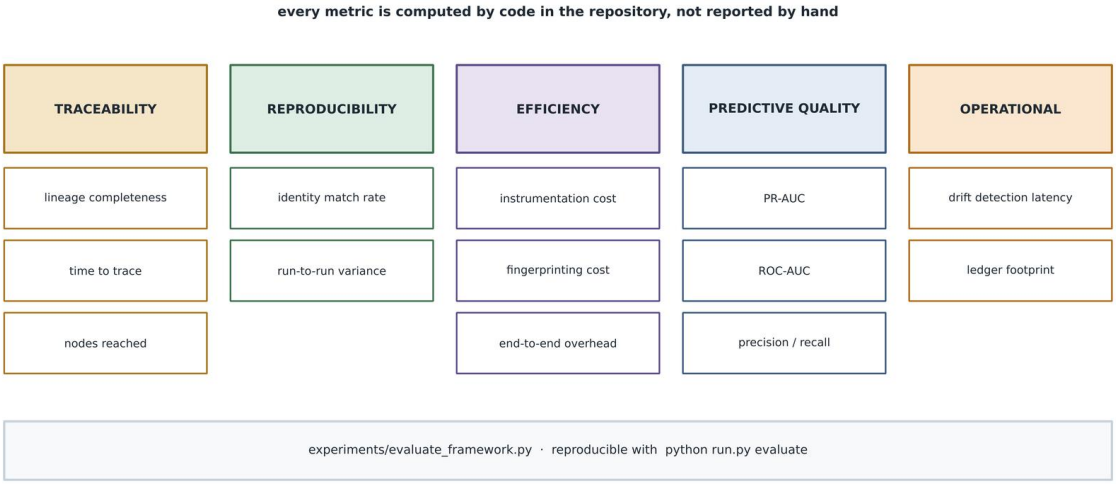


FIGURE 21. Evaluation framework: five metric families, all computed in code.

Lineage completeness is the fraction of terminal artifacts, meaning models, deployments, prediction batches and drift reports, from which an upstream traversal reaches a raw dataset node, as given in equation (7). A value below one indicates that the thread is broken somewhere.

The reproducibility rate is the fraction of independent runs whose content-addressed artifact identifiers match those of a reference run, over the set of datasets, feature sets and models, as given in equation (8).

Trace time is the median wall clock over twenty trials required to answer a provenance question for a given artifact. The baseline has no equivalent, and this is reported as not applicable rather than as a large number, because the honest statement is that the question cannot be answered from the control arm's record at all, not that answering it is slow.

Overhead is reported in two distinct ways, and the distinction is the methodological contribution of this section. Instrumentation cost is the time spent inside ledger write operations as a proportion of the wall clock of the stages the baseline also performs. End-to-end overhead is the comparison of that same restricted stage set against the baseline script. The two differ because the threaded pipeline also persists intermediate artifacts, profiles and feature specifications that the baseline holds in memory and discards.

Production monitoring platforms typically compute a family of such statistics [55]; the population stability index was chosen here for its interpretability and its established thresholds. It is defined in equation (9) over quantile bins of the reference distribution, where e and a denote the expected and actual proportion in bin i.

PSI = sum over i of (

Model quality is reported as average precision, area under the receiver operating characteristic curve, F1, precision and recall. Average precision is the primary metric for the reasons given in Section IX. Table 11 defines every metric and records the subsection in which each is reported.

TABLE 11. METRICS DEFINITION MATRIX

Metric	Definition	Reported in
Lineage completeness	Fraction of terminal artifacts reaching raw data	Section XVI-B
Reproducibility rate	Fraction of runs with matching artifact identities	Section XVI-B
Trace time	Median wall clock to answer a provenance query	Section XVI-A
Provenance depth	Number of artifacts recovered by one traversal	Section XVI-A
Instrumentation cost	Ledger write time over comparable-stage wall clock	Section XVI-C
Fingerprinting cost	Content digest time over comparable-stage wall clock	Section XVI-C
End-to-end overhead	Threaded comparable stages against baseline script	Section XVI-C
Ledger footprint	Bytes on disk for the resulting graph	Section XVI-C
Drift magnitude	Population stability index per monitored feature	Section XVI-D
Model quality	Average precision, ROC-AUC, F1, precision, recall	Section XVI-E

XVI. RESULTS AND FINDINGS

All results in this section were produced by a single command in the repository and are recorded in a machine-readable report alongside the figures.

A. TRACEABILITY PERFORMANCE

Figure 22 presents the traceability results. The median time to answer a provenance question is 0.35 milliseconds for a deployed model, 0.57 milliseconds for a prediction batch and 0.68 milliseconds for a drift report, over twenty trials each. The corresponding provenance depths are nine, thirteen and fifteen artifacts. The increase in both quantities across the three artifact classes is expected and informative: a drift report sits further downstream than a model, so more of the lifecycle lies between it and the raw data.

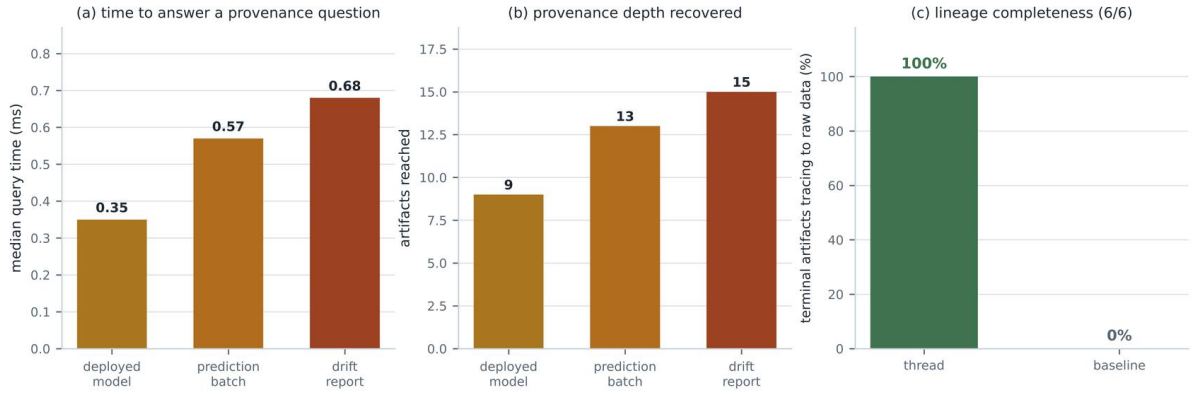


FIGURE 22. Traceability performance results, measured over twenty trials per artifact class.

The comparison with the baseline is not a comparison of magnitudes. The control arm records no lineage, so the same question cannot be posed to it. In practice an engineer would answer it by reading the script, inspecting file modification times and consulting whoever ran it, a process measured in hours or days and with no guarantee of correctness. Reporting that as a number would imply a precision that does not exist, so it is reported as not applicable.

B. REPRODUCIBILITY AND COMPLETENESS

Across five independent runs into fresh ledgers, all five artifact identifiers matched those of the reference run in every case, giving a reproducibility rate of one hundred per cent. This result should be read with its mechanism in mind. It does not say that machine learning is deterministic; it says that when the inputs, the code and the seed are held constant, the framework's content-addressed identities are stable, and therefore that any future divergence would be detected rather than absorbed silently.

Lineage completeness was one hundred per cent, with all six terminal artifacts tracing back to a raw dataset node. The corresponding figure for the control arm is zero, not because its pipeline is worse but because it records nothing to traverse.

C. LIFECYCLE EFFICIENCY

Figure 23 decomposes the cost of carrying the thread. Writing lineage consumed 0.073 seconds across 82 ledger writes, which is 1.9 per cent of the 3.85 seconds spent on the stages the baseline also performs. Content fingerprinting consumed a further 0.021 seconds over 248 digest operations, or 0.5 per cent. The ledger occupied 60 kilobytes on disk for a graph of 32 nodes and 50 edges.

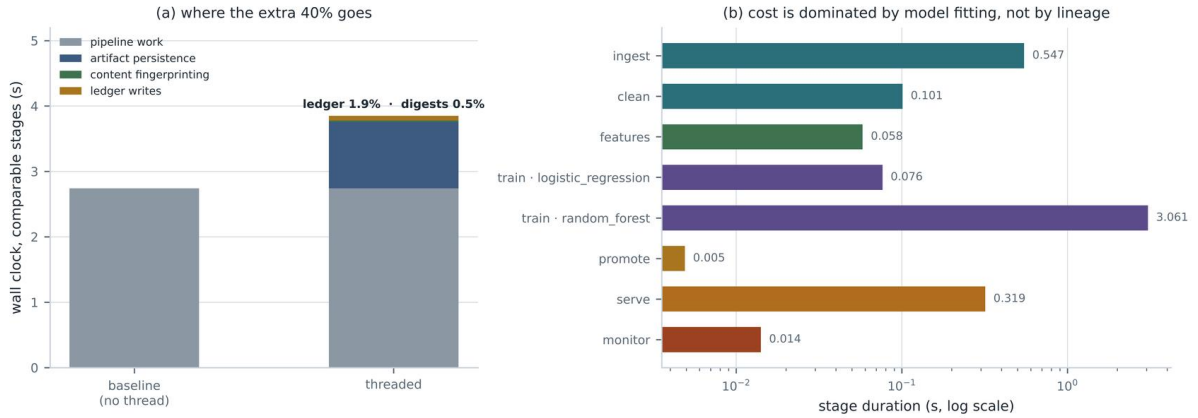


FIGURE 23. Lifecycle efficiency analysis: cost decomposition and per-stage timings.

The end-to-end comparison against the baseline script shows the threaded pipeline taking 3.85 seconds against 2.74 seconds, a difference of 40.5 per cent. Reporting only this number would be misleading, and reporting only the 1.9 per cent would be self-serving. The gap between the two is accounted for by persisting intermediate artifacts, profile reports and feature specifications that the baseline computes in memory and discards. That persistence is a precondition of traceability rather than an accident of implementation, so it belongs in the accounting; but it should be named for what it is rather than attributed to the ledger.

The right-hand panel of Figure 23 explains why the percentage is less alarming than it appears. Stage durations span three orders of magnitude, and the pipeline is dominated by fitting the random forest at 3.06 seconds. On a workload with realistic training times, the fixed cost of lineage capture becomes proportionally negligible. The 40 per cent figure is a property of a deliberately small case study, and the honest generalisation is the absolute cost of 0.073 seconds plus 60 kilobytes, not the ratio.

A methodological caution applies. During development, an earlier version of this measurement summed stage timings across the entire ledger rather than across the run under evaluation, and consequently double-counted whenever a second pipeline run was present, reporting an overhead of 137 per cent. The defect was found by noticing that the reported figure exceeded the measured wall clock of the pipeline itself. It is recorded here because it illustrates a general hazard: a self-measuring system can produce confident nonsense, and the only defence is to check the instrument against an independent quantity.

D. GOVERNANCE EFFECTIVENESS

Figure 24 reports the governance results. All five audit questions posed in Section XII are answerable from the threaded record and none from the baseline. On the monitored batch, the population stability index reached 2.561 on tool wear and 1.062 on the derived wear-torque feature, both far above the 0.20 threshold, while the four unshifted variables registered exactly zero. The monitor consequently wrote the retrain relation back to the raw dataset node, closing the loop.

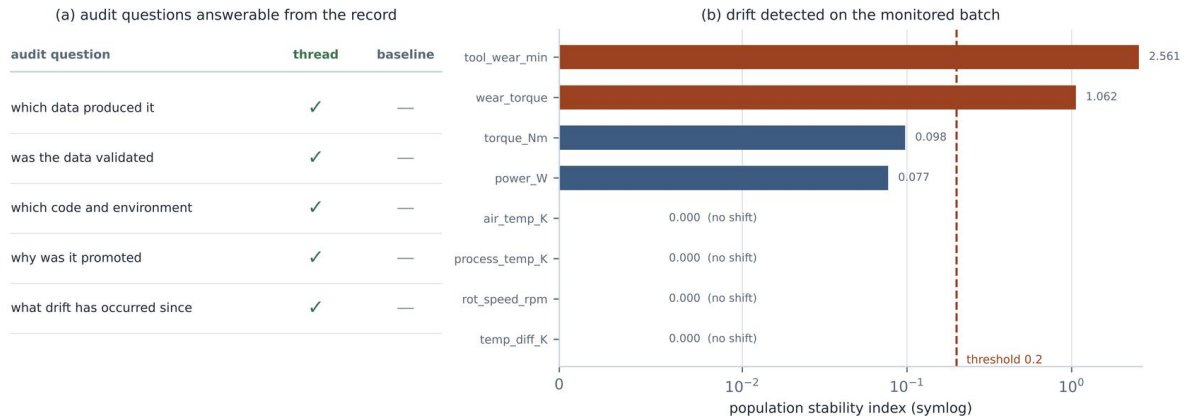


FIGURE 24. Governance effectiveness analysis: audit coverage and detected drift.

The drift was induced by ageing the monitored batch, adding sixty minutes of tool wear and scaling torque by 1.08, applied only to the served batch and never to the training data. The simulation parameters are recorded on the drift report node, so the experiment is honest about being simulated in the record itself and not only in the prose.

E. CASE STUDY MODEL QUALITY

Figure 34 and Table 12 report the predictive results. The random forest achieved an average precision of 0.735 and an area under the receiver operating characteristic curve of 0.959, against 0.269 and 0.824 for the logistic regression, and was promoted automatically by the configured gate.

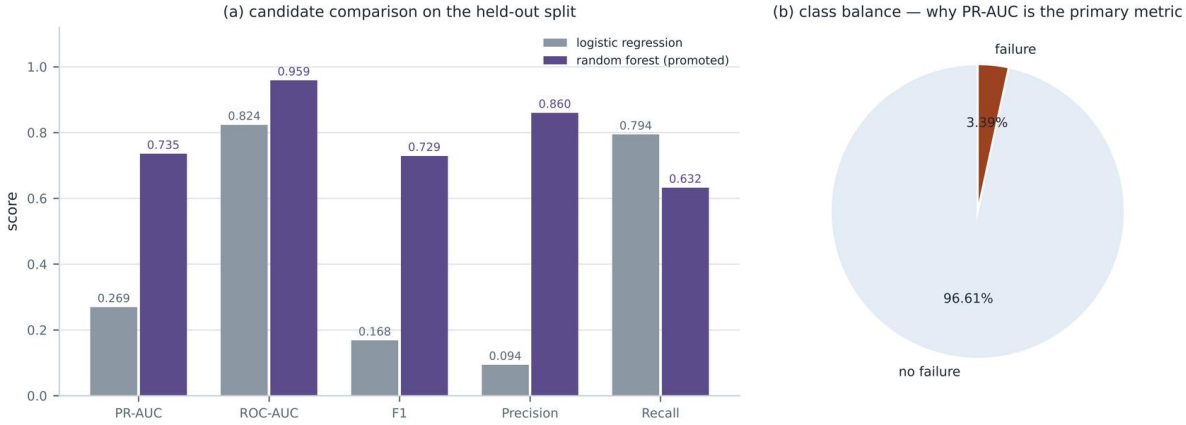


FIGURE 34. Case study model performance: candidate comparison and class balance.

The contrast between the two candidates is instructive beyond the leaderboard. Logistic regression with balanced class weights achieves high recall, at 0.794, by flagging far too much: its precision of 0.094 means roughly nine false alarms for every true one. The random forest trades recall down to 0.632 in exchange for precision of 0.860. On accuracy the two would appear comparable; on average precision, the metric the gate actually reads, the random forest is better by a factor of nearly three. This is the practical argument for the metric choice defended in Section IX [43].

TABLE 12. EXPERIMENTAL RESULTS SUMMARY

Measure	With the thread	Baseline (no thread)
Reproducibility rate over 5 runs	100 per cent	Not measurable
Lineage completeness	100 per cent (6 of 6 artifacts)	0 per cent
Trace a deployed model	0.35 ms, 9 artifacts recovered	Manual reconstruction
Trace a prediction batch	0.57 ms, 13 artifacts recovered	Manual reconstruction
Trace a drift report	0.68 ms, 15 artifacts recovered	Manual reconstruction
Ledger writes	82 writes, 0.073 s	Not applicable
Instrumentation cost	1.9 per cent of comparable runtime	Not applicable
Fingerprinting cost	0.5 per cent of comparable runtime	Not applicable
End-to-end, like for like	3.85 s (+40.5 per cent)	2.74 s

Measure	With the thread	Baseline (no thread)
Ledger footprint	60 KB, 32 nodes, 50 edges	Not applicable
Random forest PR-AUC	0.735	0.735
Logistic regression PR-AUC	0.269	0.269
Drift detected (PSI tool wear)	2.561 against threshold 0.20	Not detected

The final three rows of Table 12 are the point of the whole comparison. Predictive quality is identical in both arms, because the thread does not change the modelling. What differs is everything to do with knowing why.

XVII. COMPARATIVE ANALYSIS

Figure 25 compares the framework against four alternatives across seven capability dimensions. The chart reports capability coverage rather than measured throughput, and is labelled as such, because a fair throughput comparison against production platforms would require deploying them at equivalent scale, which was outside the scope of this study.



FIGURE 25. Comparative benchmark analysis across seven capability dimensions.

The pattern is more interesting than the ranking. Data versioning systems score well on their own axis and nowhere else. Experiment tracking systems score well on tracking and partially on serving lineage, because recent versions link datasets to logged models, but record nothing about validation or drift. The combination of the two scores well on both their axes without producing any coverage on the three axes that concern crossing between stages, which is precisely the gap argued for in Section III. The framework presented here scores lower than a dedicated data versioning system on its own axis, since it stores digests rather than managing content-addressed storage, and that is an accurate representation of the design: it is a layer over such tools rather than a replacement for them. Table 13 records the same comparison in tabular form.

TABLE 13. COMPARATIVE BENCHMARK ANALYSIS

Dimension	Conventional	DVC only	MLflow only	DVC + MLflow	This work
Data versioning	None	Full	Partial	Full	Good
Experiment tracking	None	None	Full	Full	Full
Validation outcomes in the graph	None	None	None	Partial	Full
Serving lineage	None	None	Partial	Partial	Full
Drift linked to data	None	None	None	None	Full
Bidirectional traversal	None	None	None	None	Full
Immutable history	None	Partial	None	Partial	Full

XVIII. RELIABILITY, REPRODUCIBILITY AND AUDITABILITY ASSESSMENT

Figure 26 proposes a five-level ladder for lifecycle reproducibility, from R0, where nothing is recorded, to R4, where the identity of every artifact is itself verifiable from its content. The framework presented here is evaluated at R4, and the distinction from R3 is worth drawing carefully. At R3 a run is reproducible because its inputs are pinned by reference: a dataset path, a commit hash, a requirements file. At R4 the pinning is by content, so a reference that has been silently repointed is detectable. The difference matters in exactly the situations where reproducibility claims are most often tested, namely when something has gone wrong and someone suspects that an input is not what it is said to be.

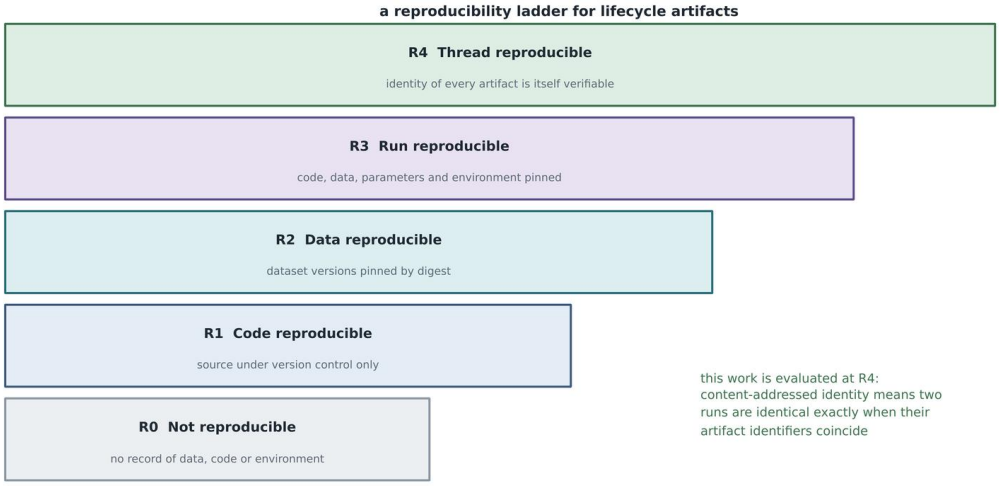


FIGURE 26. Reproducibility assessment model: a five-level ladder for lifecycle artifacts.

Reliability of the framework itself is addressed by a suite of twenty-five automated tests, which fall into three groups. Ledger tests assert the load-bearing properties directly: that identical content yields identical identifiers, that update and delete raise integrity errors, that unknown vocabulary is rejected, that duplicate edges are idempotent, that traversal recovers the full chain in both directions, that a failing stage still writes a failure outcome, and that a deliberately constructed cycle does not hang the traversal. Pipeline tests assert the stage logic against synthetic frames, including that engineered features match their symbolic definitions and that the drift statistic is zero for identical distributions and grows with a real shift. Interface tests assert that the service returns a probability with its lineage attached, that an impossible reading is rejected, that a worn tool scores higher than a fresh one, and that a trace from the deployment reaches raw data.

The third group includes one behavioural assertion rather than a structural one, and it was included deliberately. A test that merely checks the response schema would pass against a model that returns a constant. Asserting that a worn tool scores higher than a fresh one checks that the deployed artifact retains the domain relationship the features were designed to encode, which is the kind of regression that schema tests do not catch.

Table 14 records each reliability property, the mechanism that provides it and the test that demonstrates it. Auditability was assessed against the five questions enumerated in Section XII, with the result reported in Figure 24. A sixth property, exportability, is satisfied by the full graph export to JSON and to a self-contained HTML rendering produced on every pipeline run. The HTML file embeds no external resources, so it can be archived as evidence and opened years later without the framework.

TABLE 14. LIFECYCLE RELIABILITY MATRIX

Property	Mechanism	Test evidence
Identity stability	Content-addressed node identifiers	Identical content yields the same identifier
Immutability	Database triggers on both tables	Update and delete raise integrity errors
Vocabulary integrity	Validation of node and edge types on write	Unknown types are rejected
Idempotence	Uniqueness constraint on source, target and relation	Repeated edges return the existing identifier
Traversal correctness	Uniform edge direction with visited set	Full chain recovered upstream and downstream
Failure visibility	Outcome node written in a finally block	A raising stage still records a failure outcome
Cycle safety	Visited set in breadth-first traversal	A constructed cycle terminates
Serving correctness	Features recomputed on the serving path	A worn tool scores above a fresh one

XIX. INDUSTRY USE CASES AND CASE STUDY EVALUATION

Figure 27 shows how the framework maps onto a smart manufacturing environment. Four horizontal bands run from the shop floor through plant systems and analytics to decision, and the thread spans all four, which is precisely the span that today's records do not cover: machine controllers, manufacturing execution systems, historians and maintenance management systems each keep their own history in their own identifiers.

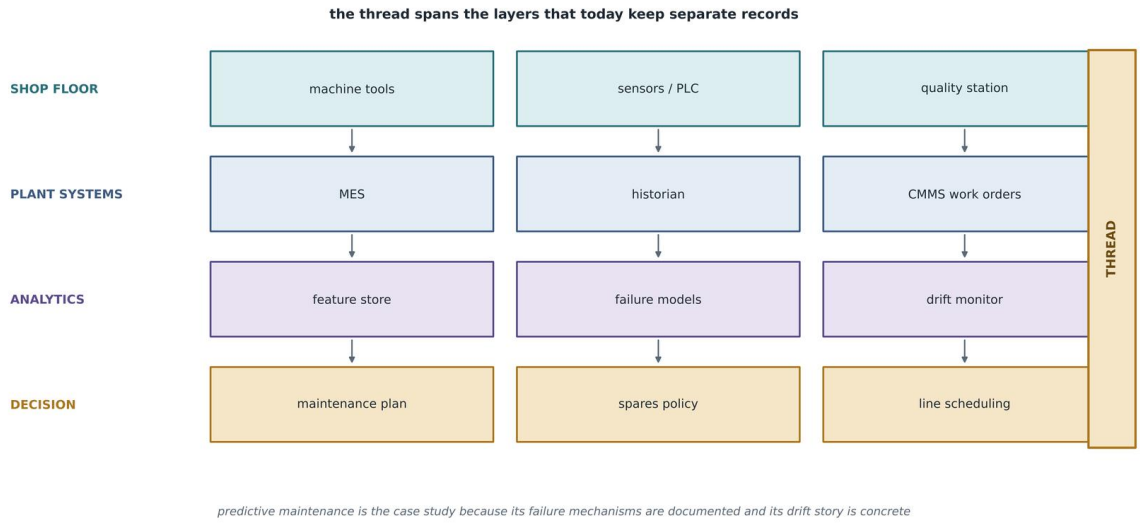


FIGURE 27. Smart manufacturing digital thread ecosystem.

The predictive maintenance case study exercised in this paper sits in the analytics band of that figure and consumes data that would, in a real deployment, arrive from the two bands below it. It was chosen because its failure mechanisms are documented, which makes the engineered features defensible rather than arbitrary, and because tool wear supplies a concrete drift narrative. Systematic reviews of the field report that condition monitoring and remaining-useful-life estimation dominate published applications, with data integration across plant systems repeatedly identified as the practical obstacle rather than the modelling itself [39], [40]. The sensing and connectivity substrate that such deployments assume is surveyed in the industrial Internet of Things literature [42], and privacy-

preserving variants have been proposed where data cannot leave the plant [41].

Figure 28 shows the aerospace variant, which is where the digital thread concept originated and where its evidential standard is highest [3]. Design, certification, manufacture, in-service operation and sustainment each generate artifacts that must remain attributable for decades. The analytics layer in such an environment inherits the configuration control of the surrounding thread rather than inventing its own, which is an argument for the design position taken in Section XI: the lineage record should not be the property of any one tool.



FIGURE 28. Aerospace lifecycle analytics architecture.

Three further domains are discussed as design rather than as results, and are marked as such in Table 15. In finance, model risk management regimes already require documented model inventories and validation evidence, and a lineage ledger supplies the inventory as a by-product. In supply chain management, traceability requirements have driven substantial work on blockchain-based provenance, and the vocabulary developed here is compatible with that work in the sense that both record custody and derivation [47], [48]. In smart cities and public-sector analytics, the governing concern is accountability for automated decisions affecting citizens, which maps onto the explainability requirement of Section IV.

It is worth stating the limit of these claims explicitly. No deployment was performed in any of these three domains, and nothing in Section XVI should be read as evidence about them. They are included because the architecture was designed with them in view, and because omitting them would misrepresent the scope the design targets.

TABLE 15. INDUSTRY USE CASES MATRIX

Domain	Primary driver	Thread artifacts of interest	Status in this study
Smart manufacturing	Unplanned downtime cost	Sensor datasets, failure models, drift reports	Implemented and evaluated
Aerospace sustainment	Certification evidence over decades	As-built records, inspection data, life models	Design analysis only
Financial services	Model risk management obligations	Model inventory, validation evidence, approval records	Design analysis only
Supply chain	Custody and origin traceability	Shipment records, derivation chains	Design analysis only
Public sector analytics	Accountability for automated decisions	Decision records, data origin, appeal evidence	Design analysis only

XX. BUSINESS IMPACT ASSESSMENT

Figure 29 separates the business impact claims this study can support from those it cannot, and the separation is the substance of the section.



FIGURE 29. Business impact assessment framework: measured categories and those requiring field study.

Four categories are supported by measurement. Debugging effort is reduced from manual reconstruction to a sub-millisecond query, with the provenance depth recovered reported in Section XVI-A. Audit preparation is reduced from evidence assembly to a traversal, with five of five audit questions answerable from the record. Reproducibility risk is addressed with a measured identity match rate across five runs. Compute overhead is quantified and decomposed rather than asserted.

Two categories are deliberately left unquantified. Downtime avoided depends on the base rate of failures, the cost of an unplanned stoppage and the operating point chosen, none of which can be obtained without deployment on a real production line. Compliance cost reduction depends on an organisation's existing audit cost baseline. Supplying figures for either would require assumptions that the study has no basis for, and a plausible number with no basis is worse than an acknowledged gap, because it is harder to detect. The broader literature on analytics capability and firm performance is clear that realised value depends on organisational factors well beyond the technical artifact [49], which reinforces the point.

The honest summary is that this study demonstrates a capability and measures its cost, and that converting the capability into a monetary figure is a separate piece of work requiring a field deployment.

XXI. CHALLENGES AND LIMITATIONS

Figure 30 maps four limitations against the work each implies.

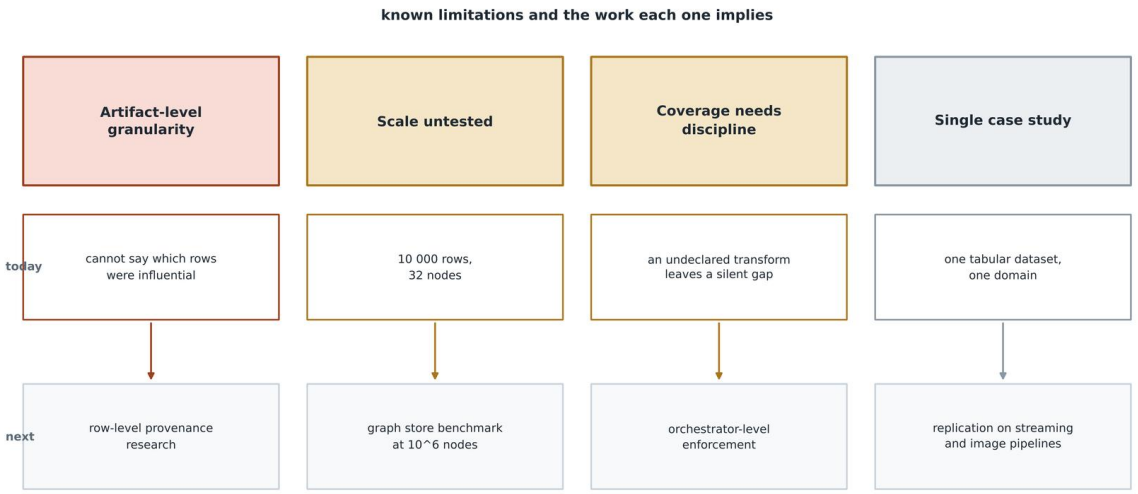


FIGURE 30. Challenges and limitations roadmap.

The first is granularity. Lineage is recorded at artifact level: this dataset version, this feature set, this model. The framework can state which dataset version produced a prediction; it cannot state which rows within

it were influential. Row-level provenance is a substantially harder problem, both conceptually and in storage terms, and it is explicitly out of scope. It is named here because it is the first question a sceptical examiner should ask.

The second is scale. The results reported here come from a ten-thousand-row dataset and a graph of thirty-two nodes. SQLite and in-memory breadth-first traversal are comfortable at that size and will remain so for a long time, but the framework has not been tested at the scale where a dedicated graph store would earn its place. The comparative literature on graph and relational database performance suggests the crossover depends strongly on query shape rather than on node count alone [44], [45], [46], which means the question cannot be settled by extrapolation and requires measurement. Establishing where the embedded store stops being adequate is the single most likely source of an unwelcome result.

The third is that coverage depends on discipline. The thread records what the stages tell it. A stage that performs an undeclared transformation produces a gap that the completeness metric will not detect, because that metric verifies that a path to raw data exists and not that the path is exhaustive. This is a genuine weakness of the evaluation and not merely of the implementation. Orchestrator-level enforcement, in which a task that does not open a ledger transaction is refused, is the intended mitigation and is not yet implemented.

The fourth is the single case study. One tabular dataset in one domain, with a simulated rather than observed drift event, supports a claim about feasibility and cost but not about generality. Replication on a streaming pipeline and on an image pipeline would test whether the eleven-type vocabulary is sufficient or merely sufficient here.

A fifth limitation concerns the data itself and has been stated in Sections XII and XIV: the reported experiments ran against a specification replica rather than the original dataset file, because the execution environment could not reach the repository. The modelling results in Section XVI-E should therefore be read as characterising the replica, and must be regenerated against the original before being cited as results on AI4I 2020.

XXII. FUTURE RESEARCH DIRECTIONS

Figure 31 organises the forward programme into three horizons, and Figure 32 sketches the end state towards which they point.

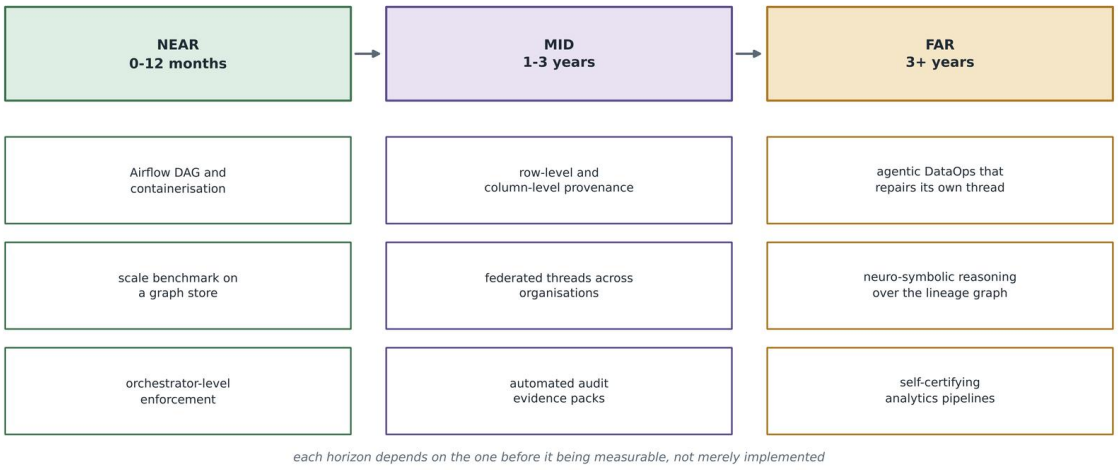


FIGURE 31. Future research roadmap across three horizons.

In the near term, three items are well defined. The orchestration layer should be completed with an Airflow directed acyclic graph alongside the existing Prefect flow, and the service containerised. The scalability question raised in Section XXI should be settled by benchmarking the traversal workload against a dedicated graph store across a range of graph sizes. And orchestrator-level enforcement should be implemented so that coverage no longer depends on developer discipline.

In the medium term, three directions follow. Row-level and column-level provenance would address the granularity limitation, most plausibly by recording transformation semantics rather than by materialising per-row lineage. Federated threads, in which organisations exchange verifiable lineage claims without exposing underlying data, would extend the model across organisational boundaries, and the supply chain traceability literature offers relevant mechanisms [47], [48]. Automated audit evidence packs, generated from the graph against a named regulatory schema, would convert the exportability property into a deliverable.

In the longer term, two research directions are worth naming. Agentic data operations, in which an automated agent detects and repairs breaks in its own thread, would change the enforcement problem from a static constraint into a control loop. And neuro-symbolic reasoning over the lineage graph would allow questions to be posed that are currently inexpressible, such as which design

decision most plausibly explains a sustained degradation; the survey literature on reasoning over knowledge graphs supplies the starting point [12], [50].



FIGURE 32. Next-generation digital thread analytics vision.

The organising idea of Figure 32 is that the thread stops being documentation and becomes a control surface. Each capability shown depends on the record being complete and trustworthy, which is why the near-term items are prerequisites rather than conveniences.

XXIII. CONCLUSION

This case study has argued that the central weakness of contemporary data science practice is not a shortage of tools but the absence of a record that joins them, and has adapted the digital thread concept from manufacturing to address it. Figure 33 summarises the four kinds of contribution the work makes.

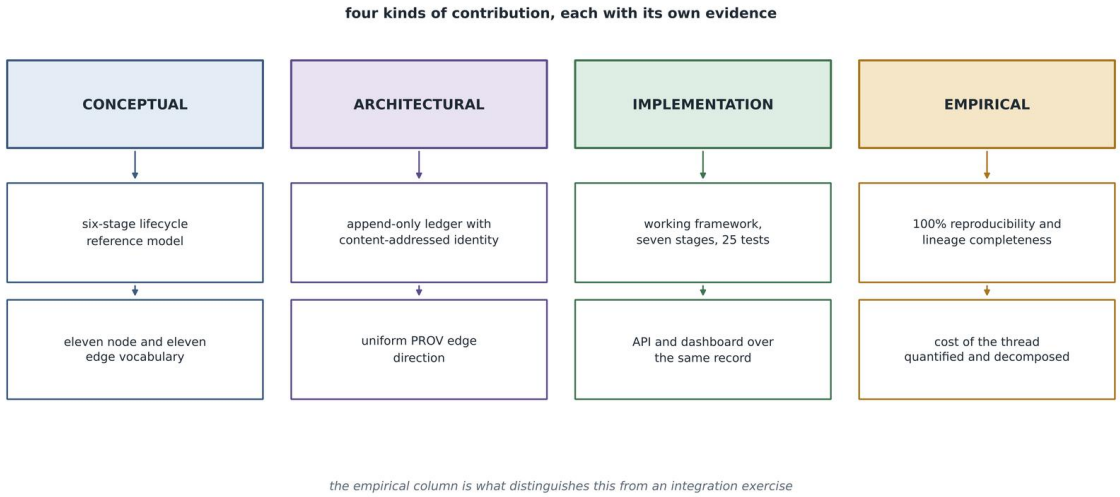


FIGURE 33. Research contribution framework.

Conceptually, the lifecycle has been formalised through a digital-thread lens, and a compact vocabulary of eleven artifact types and eleven relation types has been shown sufficient to describe a complete pass of it. Architecturally, three decisions carry the design: content-addressed artifact identity, which makes reproducibility measurable rather than assertable; append-only storage enforced at the engine rather than by convention, which makes the record evidential; and a single uniform edge direction borrowed from the W3C PROV model, which makes one traversal function correct for the whole graph. In implementation terms, a working framework of seven pipeline stages, a query interface, a visualiser, a prediction service and a dashboard has been delivered, covered by twenty-five automated tests.

Empirically, the framework achieved a reproducibility rate of one hundred per cent over five independent runs and lineage completeness of one hundred per cent, answering in under a millisecond provenance questions that the control arm cannot answer at all, at an instrumentation cost of 1.9 per cent of runtime and sixty kilobytes of storage. The end-to-end difference of 40.5 per cent has been decomposed rather than hidden, and attributed principally to artifact persistence rather than to lineage capture.

Two things this work does not establish deserve restating in closing. It does not establish behaviour at scale, since the evaluated graph contained thirty-two nodes. And it does not establish monetary business value, since no field deployment was performed. What it does establish is that end-to-end lifecycle traceability is achievable with a small, tool-agnostic abstraction, that its cost can be measured and decomposed honestly, and that the resulting record answers questions which conventional pipelines leave to memory and inference.

XXIV. DATA AVAILABILITY STATEMENT

The datasets analysed during the current study are available in the UCI Machine Learning Repository, at the persistent identifier doi:10.24432/C5HS5C [51]. The experiments reported in Section XVI were executed against a specification replica generated from that dataset's published description, because the execution environment could not reach the repository; the replica generator is included in the source code and the substitution is recorded as an attribute on the dataset node in every run. All source code, configuration, figure-generating scripts, test suites, and machine-readable evaluation reports generated during this study are openly available on GitHub at https://github.com/adminsharpcode/digital-thread-analytics. The interactive prediction service and digital thread lineage explorer dashboard is deployed live on Vercel at https://digital-thread-analytics.vercel.app.

XXV. REFERENCES

[1] T. A. Abdel-Aty and E. Negri, "Conceptualizing the digital thread for smart manufacturing: a systematic literature review," J. Intell. Manuf., vol. 35, pp. 3629-3653, 2024, doi: 10.1007/s10845-024-02407-1.

[2] Q. Zhang, J. Liu, and X. Chen, "A literature review of the digital thread: Definition, key technologies, and applications," Systems, vol. 12, no. 3, Art. no. 70, 2024, doi: 10.3390/systems12030070.

[3] N. Eskue, "Digital thread roadmap for manufacturing and health monitoring the life cycle of composite aerospace components," Aerospace, vol. 10, no. 2, Art. no. 146, 2023, doi: 10.3390/aerospace10020146.

[4] T. Y. Pang, J. D. Pelaez Restrepo, C.-T. Cheng, A. Yasin, H. Lim, and M. Miletic, "Developing a digital twin and digital thread framework for an 'Industry 4.0' shipyard," Appl. Sci., vol. 11, no. 3, Art. no. 1097, 2021, doi: 10.3390/app11031097.

[5] I. C. Fantozzi, A. Santolamazza, G. Loy, and M. M. Schiraldi, "Digital twins: Strategic guide to utilize digital twins to improve operational efficiency in Industry 4.0," Future Internet, vol. 17, no. 1, Art. no. 41, 2025, doi: 10.3390/fi17010041.

[6] L. R. R. da Silva, D. Y. Pimenov, R. B. da Silva, A. Ercetin, and K. Giasin, "Review of applications of digital twins and Industry 4.0 for machining," J. Manuf. Mater. Process., vol. 9, no. 7, Art. no. 211, 2025, doi: 10.3390/jmmp9070211.

[7] G. Amirkhanova, S. Adilkyzy, B. Amirkhanov, D. Baizhanova, and S. Chen, "A digital twin architecture for integrating lean manufacturing with industrial IoT and predictive analytics," Information, vol. 17, no. 2, Art. no. 196, 2026, doi: 10.3390/info17020196.

[8] L. Moreau and P. Missier, Eds., PROV-DM: The PROV Data Model, W3C Recommendation, Apr. 30, 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[9] M. Hofer, D. Obraczka, A. Saeedi, H. Koepcke, and E. Rahm, "Construction of knowledge graphs: Current state and challenges," Information, vol. 15, no. 8, Art. no. 509, 2024, doi: 10.3390/info15080509.

[10] M. A. Sadiq, P. K. Langat, and A. Neupane, "GeoPROV: A domain-specialised provenance model for spatial data supply chains," ISPRS Int. J. Geo-Inf., vol. 15, no. 6, Art. no. 272, 2026, doi: 10.3390/ijgi15060272.

[11] V. Kalokyri et al., "AI model passport: Data and system traceability framework for transparent AI in health," arXiv:2506.22358, 2025.

[12] L. N. DeLong, R. Fernandez Mir, and J. D. Fleuriot, "Neurosymbolic AI for reasoning over knowledge graphs: A survey," arXiv:2302.07200, 2023.

[13] D. Kreuzberger, N. Kuehl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," arXiv:2205.02302, 2022.

[14] B. Eken, S. Pallewatta, N. K. Tran, A. Tosun, and M. A. Babar, "A multivocal review of MLOps practices, challenges and open issues," arXiv:2406.09737, 2024.

[15] M. A. Mateo-Casali, A. Boza, and F. Fraile, "Towards a reference architecture for machine learning operations," Computers, vol. 15, no. 4, Art. no. 218, 2026, doi: 10.3390/computers15040218.

[16] P. Ruf, M. Madan, C. Reich, and D. Ould-Abdeslam, "Demystifying MLOps and presenting a recipe for the selection of open-source tools," Appl. Sci., vol. 11, no. 19, Art. no. 8861, 2021, doi: 10.3390/app11198861.

[17] L. Berberi, V. Kozlov, G. Nguyen et al., "Machine learning operations landscape: Platforms and tools," Artif. Intell. Rev., vol. 58, Art. no. 167, 2025, doi: 10.1007/s10462-025-11164-3.

[18] R. Minon et al., "ArtifactOps and ArtifactDL: A methodology and a language for conceptualizing and operationalising different types of pipelines," J. Cloud Comput., vol. 14, no. 1, Art. no. 42, Aug. 2025, doi: 10.1186/s13677-025-00761-w.

[19] I. Prapas, B. Derakhshan, A. R. Mahdiraji et al., "Continuous training and deployment of deep learning models," Datenbank Spektrum, vol. 21, pp. 203-212, 2021, doi: 10.1007/s13222-021-00386-8.

[20] H. Semmelrock et al., "Reproducibility in machine learning-based research: Overview, barriers and drivers," arXiv:2406.14325, 2024.

[21] M. Mitchell et al., "Model cards for model reporting," arXiv:1810.03993, 2018.

[22] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daume III, and K. Crawford, "Datasheets for datasets," arXiv:1803.09010, 2018.

[23] L. Orr, A. Sanyal, X. Ling, K. Goel, and M. Leszczynski, "Managing ML pipelines: Feature stores and the coming wave of embedding ecosystems," arXiv:2108.05053, 2021.

[24] A. Li et al., "Managed geo-distributed feature store: Architecture and system design," arXiv:2305.20077, 2023.

[25] G. Hovakimyan and J. M. Bravo, "Evolving strategies in machine learning: A systematic review of concept drift detection," Information, vol. 15, no. 12, Art. no. 786, 2024, doi: 10.3390/info15120786.

[26] D. Lukats, O. Zielinski, A. Hahn et al., "A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams," Int. J. Data Sci. Anal., vol. 19, no. 1, pp. 1-31, 2025, doi: 10.1007/s41060-024-00620-y.

[27] H. Wiemer, A. Dementyev, and S. Ihlenfeldt, "A holistic quality assurance approach for machine learning applications in cyber-physical production systems," Appl. Sci., vol. 11, no. 20, Art. no. 9590, 2021, doi: 10.3390/app11209590.

[28] I. Blohm, F. Wortmann, C. Legner et al., "Data products, data mesh, and data fabric," Bus. Inf. Syst. Eng., vol. 66, no. 5, pp. 643-652, 2024, doi: 10.1007/s12599-024-00876-5.

[29] A. Dolhopolov, A. Castelltort, and A. Laurent, "Implementing federated governance in data mesh architecture," Future Internet, vol. 16, no. 4, Art. no. 115, 2024, doi: 10.3390/fi16040115.

[30] A. Giess and A. Hutterer, "The future of data management: A delimitation of data platforms, data spaces, data meshes, and data fabrics," Inf. Syst. E-Bus. Manage., vol. 23, pp. 971-997, 2025, doi: 10.1007/s10257-025-00707-4.

[31] C. Tlemcani, A. Z. Faroukhi, and Y. Gahi, "From ETL to modern data stack: A systematic review and strategic implementation framework," Information, vol. 17, no. 7, Art. no. 630, 2026, doi: 10.3390/info17070630.

[32] W. Elouataoui and Y. Gahi, "Empirical evaluation of big data stacks: Performance and design analysis of Hadoop, modern, and cloud architectures," Big Data Cogn. Comput., vol. 10, no. 1, Art. no. 7, 2026, doi: 10.3390/bdcc10010007.

[33] A. Goncalves and A. Correia, "XAI-compliance-by-design: A modular framework for GDPR- and AI Act-aligned decision transparency in high-risk AI systems," J. Cybersecur. Priv., vol. 6, no. 2, Art. no. 43, 2026, doi: 10.3390/jcp6020043.

[34] M. Achouch et al., "On predictive maintenance in Industry 4.0: Overview, models, and challenges," Appl. Sci., vol. 12, no. 16, Art. no. 8081, 2022, doi: 10.3390/app12168081.

[35] A. Ucar, M. Karakose, and N. Kirimca, "Artificial intelligence for predictive maintenance applications: Key components, trustworthiness, and future trends," Appl. Sci., vol. 14, no. 2, Art. no. 898, 2024, doi: 10.3390/app14020898.

[36] C. Tsallis, P. Papageorgas, D. Piromalis, and R. A. Munteanu, "Application-wise review of machine learning-based predictive maintenance: Trends, challenges, and future directions," Appl. Sci., vol. 15, no. 9, Art. no. 4898, 2025, doi: 10.3390/app15094898.

[37] J. Paredes, D. Chavez, R. Isa-Jara, and D. Vargas, "Hybrid approaches of machine learning algorithms in predictive maintenance: A systematic literature review," Appl. Syst. Innov., vol. 9, no. 5, Art. no. 90, 2026, doi: 10.3390/asi9050090.

[38] F. Arena, M. Collotta, L. Luca, M. Ruggieri, and F. G. Termine, "Predictive maintenance in the automotive sector: A literature review," Math. Comput. Appl., vol. 27, no. 1, Art. no. 2, 2022, doi: 10.3390/mca27010002.

[39] J. Garcia, L. Rios-Colque, A. Pena, and L. Rojas, "Condition monitoring and predictive maintenance in industrial equipment: An NLP-assisted review of signal processing, hybrid models, and implementation challenges," Appl. Sci., vol. 15, no. 10, Art. no. 5465, 2025, doi: 10.3390/app15105465.

[40] D. Arez, H. V. G. Navas, and P. Gaspar, "The bridge between artificial intelligence and predictive maintenance in Industry 4.0: A systematic review," Appl. Sci., vol. 16, no. 10, Art. no. 4882, 2026, doi: 10.3390/app16104882.

[41] H. M. H. A. Alshkeili, S. J. Almheiri, and M. A. Khan, "Privacy-preserving interpretability: An explainable federated learning model for predictive maintenance in sustainable manufacturing and Industry 4.0," AI, vol. 6, no. 6, Art. no. 117, 2025, doi: 10.3390/ai6060117.

[42] N. Haqiq, M. Zaim, A. Haqiq, M. Sbihi, and A. El Ouaazizi, "Internet of Things for Industry 4.0: A systematic literature review of technologies, architectures, applications, and challenges," IoT, vol. 7, no. 2, Art. no. 46, 2026, doi: 10.3390/iot7020046.

[43] M. B. A. McDermott, H. Zhang, L. H. Hansen, G. Angelotti, and J. Gallifant, "A closer look at AUROC and AUPRC under class imbalance," arXiv:2401.06091, 2024.

[44] P. Kotiranta, M. Junkkari, and J. Nummenmaa, "Performance of graph and relational databases in complex queries," Appl. Sci., vol. 12, no. 13, Art. no. 6490, 2022, doi: 10.3390/app12136490.

[45] J. Monteiro, F. Sa, and J. Bernardino, "Experimental evaluation of graph databases: JanusGraph, Nebula Graph, Neo4j, and TigerGraph," Appl. Sci., vol. 13, no. 9, Art. no. 5770, 2023, doi: 10.3390/app13095770.

[46] J. Sandell, E. Asplund, W. Y. Ayele, and M. Duneld, "Performance comparison analysis of ArangoDB, MySQL, and Neo4j: An experimental study of querying connected data," arXiv:2401.17482, 2024.

[47] P. Azevedo, J. Gomes, and M. Romao, "Supply chain traceability using blockchain," Oper. Manag. Res., vol. 16, pp. 1359-1381, 2023, doi: 10.1007/s12063-023-00359-y.

[48] O. Karaduman and G. Gulhas, "Blockchain-enabled supply chain management: A review of security, traceability, and data integrity amid the evolving systemic demand," Appl. Sci., vol. 15, no. 9, Art. no. 5168, 2025, doi: 10.3390/app15095168.

[49] P. Maroufkhani, R. Wagner, W. K. Wan Ismail, M. B. Baroto, and M. Nourani, "Big data analytics and firm performance: A systematic review," Information, vol. 10, no. 7, Art. no. 226, 2019, doi: 10.3390/info10070226.

[50] B. Liang, Y. Wang, and C. Tong, "AI reasoning in deep learning era: From symbolic AI to neural-symbolic AI," Mathematics, vol. 13, no. 11, Art. no. 1707, 2025, doi: 10.3390/math13111707.

[51] S. Matzka, "AI4I 2020 predictive maintenance dataset," UCI Machine Learning Repository, 2020, doi: 10.24432/C5HS5C.

[52] MLflow Project, "MLflow documentation: Dataset tracking and logged models," 2025. [Online]. Available: https://mlflow.org/docs/latest/ml/dataset/

[53] Iterative Inc., "DVC: Data version control documentation," 2025. [Online]. Available: https://dvc.org/doc

[54] OpenLineage Project, "OpenLineage: An open standard for lineage metadata collection," 2025. [Online]. Available: https://openlineage.io/

[55] Evidently AI, "Evidently: ML model monitoring and drift detection documentation," 2025. [Online]. Available: https://docs.evidentlyai.com/

[56] Apache Software Foundation, "Apache Airflow documentation," 2025. [Online]. Available: https://airflow.apache.org/docs/